

AUTOMATED DETECTION, TRACKING AND TACTICAL ANALYSIS IN BROADCAST FOOTBALL VIDEO

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Aryan Tiwari

Supervised by Dr Aphrodite Galata

Department of Computer Science

Contents

Abstract	10
Declaration	11
Copyright	12
Acknowledgements	13
1 Introduction	14
1.1 Motivation and Context	14
1.2 Aims and Objectives	15
1.3 Evaluation Strategy	16
1.4 Report Structure	17
2 Background and Theory	19
2.1 Broadcast Football Video Analysis	19
2.2 Related Work	20
2.2.1 Commercial Tracking Systems	20
2.2.2 Academic Benchmarks and Integrated Pipelines	20
2.2.3 Positioning of This Project	21
2.3 Object Detection with YOLO	22
2.3.1 Bounding-Box Localisation and IoU	22
2.3.2 Precision, Recall, and Mean Average Precision	22
2.3.3 Detection Paradigms	23
2.3.4 The YOLO Architecture	24
2.4 Small-Object Detection and Dedicated Ball Detection	26
2.4.1 Why Small Objects Are Harder to Detect	26
2.4.2 Strategies for Small-Object Detection	27

2.4.3	Temporal Filtering of Ball Candidates	28
2.5	Multi-Object Tracking by Detection	28
2.5.1	Tracking-by-Detection	29
2.5.2	ByteTrack	30
2.5.3	BoT-SORT	32
2.5.4	Tracking Evaluation: HOTA	32
2.6	Unsupervised Team Identification from Visual Features	33
2.6.1	Approaches to Team Identification	33
2.6.2	Deep Visual Embeddings	34
2.6.3	Dimensionality Reduction and Clustering	35
2.6.4	Temporal Stability and Special Cases	36
2.7	Pitch Keypoint Detection and Planar Homography	36
2.7.1	Registration Approaches	36
2.7.2	Planar Homography	37
2.7.3	Outlier Rejection with RANSAC	38
2.7.4	Reprojection Error and Bird's-Eye Mapping	38
2.8	Spatial Analytics in Pitch Space	39
2.8.1	Player Projection onto the Pitch	39
2.8.2	Heatmap Construction	40
2.8.3	Speed and Distance Estimation	40
2.9	Event Reasoning and Match Statistics	41
2.9.1	Ball Possession Estimation	41
2.9.2	Pass Detection	42
2.9.3	Offside Rule Formalisation	43
2.10	Error Propagation in a Multi-Stage Pipeline	44
2.11	Chapter Summary	46
3	Methodology and Implementation	47
3.1	Chapter Overview	47
3.2	Requirements and Design Objectives	48
3.2.1	Functional Requirements	48
3.2.2	Non-functional Requirements	49
3.2.3	Success Criteria	49
3.3	System Architecture	50
3.3.1	Pipeline Overview	51
3.3.2	Data Flow and Intermediate Representations	52

3.3.3	System Outputs	52
3.4	Datasets and Model Training	53
3.4.1	Object Detection Dataset and Training	53
3.4.2	Ball Detection Dataset and Training	54
3.4.3	Pitch Keypoint Dataset and Training	54
3.4.4	Training Configuration	55
3.5	Detection and Tracking	55
3.5.1	Main Object Detection	55
3.5.2	Dedicated Ball Detection and Filtering	56
3.5.3	Multi-object Tracking	56
3.5.4	Track Handling and Failure Cases	57
3.6	Team Identification	57
3.6.1	Warm-up Sampling and Player Cropping	58
3.6.2	Embedding and Clustering	58
3.6.3	Temporal Smoothing and Role Handling	59
3.7	Pitch Registration	59
3.7.1	Pitch Keypoint Detection	59
3.7.2	Homography Estimation and Coordinate Transformation . . .	60
3.7.3	Caching and Robustness Measures	60
3.8	Tactical Output Generation	61
3.8.1	Bird's-eye Visualisation	61
3.8.2	Heatmaps, Speed and Distance	61
3.8.3	Possession and Pass Detection	62
3.8.4	Offside Candidate Detection	62
3.9	Evaluation Methodology	63
3.9.1	Component-level Evaluation	63
3.9.2	End-to-end Pipeline Evaluation	64
3.9.3	Ablation Studies	65
3.9.4	Runtime Evaluation	65
3.9.5	Error Analysis	66
3.10	Implementation and Reproducibility	66
3.10.1	Software Environment	67
3.10.2	Configuration and Model Weights	67
3.10.3	Running the Pipeline	68
3.10.4	Reproducibility Limitations	68

3.11 Chapter Summary	69
4 Results	70
4.1 Experimental Setup	70
4.2 Findings	70
5 Evaluation	71
5.1 Analysis	71
5.2 Discussion	71
5.3 Limitations	71
6 Conclusion	72
6.1 Summary	72
6.2 Future Work	72
A Additional Materials	78

Word Count: 12000 (TODO: update at the end)

List of Tables

1.1	Summary of evaluation metrics used in this project.	17
2.1	Comparison of object detection paradigms. Single-stage anchor-free detectors simplify training and offer competitive accuracy at high throughput, making them well suited to video-rate applications.	24
2.2	Comparison of ByteTrack and BoT-SORT. BoT-SORT adds camera-motion compensation and appearance features at the cost of additional computation, while ByteTrack achieves strong performance through its two-stage association strategy alone.	32
2.3	Comparison of representation strategies for unsupervised team identification. Vision–language model embeddings offer the best trade-off between robustness and generality, at the cost of higher-dimensional feature vectors.	34
3.1	Summary of supervised models trained for the pipeline.	55

List of Figures

- 2.1 Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$ 23
- 2.2 Generalised architecture of an anchor-free single-stage object detector. The backbone extracts feature maps at progressively lower spatial resolutions ($H/8$, $H/16$, $H/32$); the FPN neck fuses these across scales; the anchor-free head regresses bounding-box coordinates and class scores at each spatial location; and Non-Maximum Suppression produces the final detection set $\{(b_i, c_i, s_i)\}$ as defined in Equation (2.1). 25
- 2.3 Effect of pixel displacement δ on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At $\delta = 2$ pixels, a 16×16 box drops to $\text{IoU} = 0.78$ while a 64×64 box retains $\text{IoU} = 0.94$, illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard mAP_{50} threshold. . . 27
- 2.4 ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low} . Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low} . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames. 31

2.5	Error propagation paths in a broadcast football analysis pipeline. Solid arrows show the primary data flow between processing stages; dashed arrows indicate how errors at each stage propagate to downstream outputs. A detection error (e.g. a missed player) can cascade through tracking, team identification, and spatial analytics, ultimately affecting event-level outputs such as possession and offside decisions.	45
3.1	Overview of the proposed broadcast football video analysis pipeline. The system begins with object and ball detection, then applies tracking, team identification, pitch registration, and tactical analysis to produce visual and quantitative outputs.	51

List of Abbreviations

HOTA	Higher Order Tracking Accuracy
IDF1	Identification F1
IoU	Intersection over Union
MAE	Mean Absolute Error
mAP	mean Average Precision
MOTA	Multi-Object Tracking Accuracy

Abstract

TODO: my abstract (to be done at the end of the report)

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this report (including any appendices and/or schedules to this report) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this report, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the report, for example graphs and tables (“Reproductions”), which may be described in this report, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this report, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Aphrodite Galata, for her invaluable guidance, support, and advice throughout this project.

I am also grateful to Kunj Dhola and Selim Sayed for their collaboration and contributions to the project.

Finally, I would like to thank my family and friends for their continued encouragement and support throughout my studies.

Chapter 1

Introduction

1.1 Motivation and Context

The use of data and analytics in professional football has grown substantially in recent years, with clubs and national federations increasingly incorporating analytical methods into performance analysis and decision-making [26]. In particular, positional and tracking data are valuable because they provide richer descriptions of player movement, team organisation, and tactical behaviour than traditional notational analysis alone [12, 44]. As a result, access to reliable spatial match data has become increasingly important.

Despite this, high-quality football data remain difficult to access outside elite settings. Advanced event and tracking data are often proprietary, expensive, or dependent on specialist collection systems, which limits their availability for public and reproducible work [22, 9, 6]. This motivates the investigation of alternative ways to generate structured football data from more widely available sources.

Broadcast video is an attractive candidate because it is widely available and already contains much of the spatial information needed for tactical analysis. However, extracting reliable structured data from broadcast footage is challenging, since it requires several interconnected tasks such as player detection, tracking, team assignment, and pitch registration under conditions including camera motion, occlusion, scale variation, and motion blur [24, 4, 43, 42].

Recent research has shown progress on many of these components and, increasingly, on parts of the broader workflow. Open benchmark efforts such as SoccerNet-Tracking and SoccerNet Game State Reconstruction demonstrate that broadcast football video can support tracking and game-state reconstruction in realistic settings [5,

40]. However, these systems typically produce intermediate outputs such as tracked player positions or minimap reconstructions rather than directly usable tactical outputs. What remains missing is a publicly reproducible end-to-end system that integrates all major stages from broadcast video to tactical output using modern computer vision methods.

1.2 Aims and Objectives

To address this gap, the project investigates how far a modular computer vision pipeline can extract reliable tactical and spatial information from broadcast football video without requiring specialised capture systems or manual team labels.

To achieve this aim, the project pursues the following objectives:

- develop and evaluate a detection approach for the main on-pitch entity classes—players, goalkeepers, referees, and the ball—from broadcast football footage;
- assess whether multi-object tracking can preserve sufficiently consistent identities over time to support downstream tactical analysis;
- test whether unsupervised clustering of visual embeddings can separate players by team without manual team labels;
- measure the geometric accuracy of pitch keypoint detection and homography estimation for projecting image-space positions into pitch-space;
- determine whether the resulting projected trajectories can support higher-level tactical outputs, including bird’s-eye visualisations, heatmaps, possession estimates, pass events, and offside-related analysis; and
- evaluate the pipeline as an end-to-end system, including how errors in detection, tracking, and homography estimation affect downstream analytical outputs.

The project does not aim to replace professional commercial tracking systems, but rather to assess how much tactical information a reproducible, open pipeline can extract from standard broadcast footage.

The project will be considered successful if: (i) the main object detector achieves $\text{mAP}_{50} \geq 0.80$ across entity classes; (ii) multi-object tracking reaches $\text{HOTA} \geq 0.45$ on a labelled test segment; (iii) unsupervised team assignment achieves $F_1 \geq 0.85$; and

(iv) the integrated pipeline produces coherent tactical outputs—including bird’s-eye projections, heatmaps, and offside analysis—that are qualitatively consistent with the underlying match footage. These thresholds reflect realistic performance expectations for broadcast football analysis informed by the literature [43, 5].

1.3 Evaluation Strategy

This project will be evaluated at both *component level* and *system level*. This is necessary because downstream outputs such as possession estimates, heatmaps, and offside decisions depend not only on the performance of individual modules, but also on how errors propagate across detection, tracking, and pitch registration stages [43, 42]. The evaluation therefore asks two questions: whether each component performs adequately on its own task, and whether the integrated pipeline produces coherent and tactically meaningful outputs as a complete system.

At component level, each module will be assessed using standard task-appropriate metrics from the computer vision literature. At system level, the pipeline will be judged by the coherence, plausibility, and usefulness of its final outputs on representative broadcast sequences. The chosen metrics reflect the main technical risks of the project: detection accuracy, temporal identity consistency, geometric projection quality, and the validity of downstream tactical inference. These metrics were chosen because each targets a distinct failure mode of the pipeline. Mean Average Precision (mAP) is the standard object detection metric in the computer vision literature and directly measures whether the detector can reliably localise and classify entities under broadcast conditions [43]. Higher Order Tracking Accuracy (HOTA) was selected over legacy Multi-Object Tracking (MOT) metrics such as Multi-Object Tracking Accuracy (MOTA) because it balances detection and association quality in a single score, making it better suited to evaluating identity consistency over time [27]. Reprojection error measures how accurately estimated homographies map known pitch points, which determines whether projected player positions are geometrically meaningful.

At system level, quantitative metrics alone cannot capture whether tactical outputs are useful, so qualitative assessment against the source footage is included following the approach used in recent game-state reconstruction benchmarks [40]. The numeric thresholds were set based on reported performance in comparable broadcast football settings: for example, Theiner et al. [43] report mAP₅₀ values in the range 0.80–0.90 for player detection, and SoccerNet-Tracking baselines report HOTA scores

in the 0.40–0.55 range [5], making the chosen targets realistic but non-trivial. Detailed annotation procedures and experimental methodology are presented in Chapter 5.

Table 1.1: Summary of evaluation metrics used in this project.

Component / Output	Primary metrics	Purpose
Object detection	mAP _{50–95} , mAP ₅₀ , precision, recall	Measure localisation and classification performance for players, goalkeepers, referees, and the ball.
Ball detection	mAP ₅₀ , precision, recall, false positive rate	Assess robustness under small object size, motion blur, and occlusion.
Pitch keypoints and homography	mAP _{50–95} for keypoints; reprojection error for homography	Measure field registration quality and projection accuracy in pitch-space.
Multi-object tracking	HOTA, IDF1, MOTA, identity switches	Evaluate temporal consistency and identity preservation [27, 8].
Team assignment	Precision, recall, F_1 score	Measure accuracy of unsupervised team separation.
Tactical outputs	Precision, recall, and F_1 for event detection; MAE for possession; qualitative assessment for heatmaps and bird’s-eye views	Assess whether projected trajectories support useful downstream analysis.
End-to-end pipeline	Integrated case study and failure analysis	Evaluate output coherence and identify how upstream errors affect downstream reasoning.

Component-level evaluation shows whether each stage meets its technical objective, while system-level evaluation tests whether the combined pipeline remains analytically useful once these stages are linked. The final judgement of success will therefore be based on both quantitative component metrics and qualitative assessment of whether the integrated outputs remain consistent with the source footage and useful for tactical interpretation.

1.4 Report Structure

The remainder of this report is organised to guide the reader from the background knowledge needed to understand the system, through its design and outputs, to a critical assessment of its performance. Chapter 2 (*Background and Theory*) reviews the relevant literature and technical foundations, establishing the context from which the pipeline design decisions follow. Chapter 3 (*Methodology and Implementation*) describes the architecture and implementation of each pipeline stage, building directly on

the methods and models introduced in the background. Chapter 4 (*Results*) presents the outputs produced by the system in practice. Chapter 5 (*Evaluation*) then assesses the project at both component and system level, using the metrics and strategy outlined in Section 1.3, including failure analysis and the effect of error propagation on downstream outputs. Finally, Chapter 6 (*Conclusion*) summarises the main achievements, reflects on limitations, and outlines directions for future work.

Chapter 2

Background and Theory

2.1 Broadcast Football Video Analysis

Extracting structured information from broadcast football footage poses several coupled technical challenges that distinguish it from analysis based on dedicated tracking infrastructure [43, 24]. Unlike specialised systems that rely on multi-camera installations or stadium-side sensors, a broadcast pipeline must recover all player, ball, and pitch information from a single compressed, moving-camera video stream [12].

First, the broadcast camera is non-stationary: it undergoes continuous pan, tilt, and zoom in response to play, so the background cannot be treated as a fixed spatial reference and inter-frame motion cannot be attributed to scene objects alone [43]. Second, the ball is extremely small relative to the frame and is frequently blurred, occluded, or temporarily absent, making reliable detection substantially harder than for players [13]. Third, players on the same team share near-identical visual appearance due to uniform colour and similar build, so tracking and team identification must rely on subtle spatial and visual cues [43]. Fourth, image-space measurements are subject to perspective distortion, so quantities such as distance, speed, and tactical positioning are meaningful only after registration to a canonical pitch model [42].

A widely adopted response to this complexity is to decompose the problem into a sequence of specialised components [43, 24, 12]: object detection and ball detection provide the primary observations, multi-object tracking maintains temporal identity, team assignment infers player affiliation, pitch registration maps image coordinates into a canonical field representation, and higher-level outputs such as heatmaps, possession estimates, and offside reasoning are then expressed in pitch space rather than image coordinates. The remainder of this chapter reviews the theoretical basis of these

components, from detection and tracking through to registration and pitch-space analysis.

2.2 Related Work

Given the technical challenges outlined above, this section reviews existing systems and benchmarks that address broadcast football analysis at the pipeline or system level.

2.2.1 Commercial Tracking Systems

In elite professional settings, a major source of positional football data is commercial optical or sensor-based tracking. TRACAB uses multi-camera optical installations to produce high-frequency player and ball coordinates, with Linke et al. [22] reporting sub-metre positional accuracy under validated match conditions. Hudl StatsBomb provides structured event data and freeze-frame positional snapshots collected through a combination of manual annotation and automated processing, while Second Spectrum offers real-time optical tracking and tactical analysis through proprietary multi-camera systems [6]. Although these providers differ in their capture methodology and output granularity, they share a common limitation for research purposes: their underlying algorithms and raw data are not publicly available, making independent reproduction and open benchmarking difficult [9, 6].

2.2.2 Academic Benchmarks and Integrated Pipelines

The SoccerNet ecosystem has emerged as the primary open benchmark for broadcast football video understanding. SoccerNet-v2 [7] introduced large-scale annotations for action spotting, camera shot segmentation, and replay detection across hundreds of broadcast matches, establishing a shared evaluation framework for the community. SoccerNet-Tracking [5] extended this to multi-object tracking, providing bounding-box annotations and standardised metrics for player tracking in broadcast footage. Most recently, SoccerNet Game State Reconstruction [40] framed the problem as end-to-end athlete tracking and identification on a minimap, integrating detection, tracking, re-identification, and pitch registration into a single evaluated task.

Beyond SoccerNet, several studies have addressed substantial portions of the broadcast analysis workflow. Theiner et al. [43] developed a pipeline for extracting positional data from broadcast video, combining detection, tracking, and pitch registration,

but focused primarily on the accuracy of the positional output rather than on deriving tactical events from it. Magera et al. [29] proposed BroadTrack, a broadcast camera tracking system that halved reprojection error rates on the SoccerNet calibration benchmark, but addressed only the registration stage without integrating downstream player analysis. Maglo et al. [30] presented an end-to-end framework for individual player localisation from a single moving broadcast view, combining field registration with self-supervised re-identification, but evaluated the system on localisation accuracy rather than tactical output. Earlier work by Beetz et al. [4] proposed ASPOG-AMO, an automated sports game analysis system that combined camera estimation with player localisation, but predated modern deep-learning methods and did not address team identification or event-level reasoning.

A common thread across these systems is that their evaluation stops at intermediate representations. SoccerNet Game State Reconstruction, for example, measures whether players can be correctly located and identified on a top-down pitch model, but does not assess whether those reconstructions are sufficiently accurate to support tactical inference. This distinction matters because intermediate representations can score well on reconstruction metrics while still containing localisation or identity errors that are too large for reliable event-level reasoning. Within the systems reviewed above, evaluation generally does not extend to whether positional outputs can support possession estimation, pass detection, or offside analysis, which are the kinds of tactical outputs that would make a broadcast pipeline practically useful beyond reconstruction.

2.2.3 Positioning of This Project

The review above suggests that, within the literature surveyed, strong methods exist for individual components and, increasingly, for integrated reconstruction, but open systems that carry the pipeline beyond intermediate positional output to produce tactical analysis remain scarce. Commercial providers can deliver practically useful positional and tactical outputs, but the underlying systems are closed; academic systems are open but generally evaluate at the reconstruction level. Accordingly, this project investigates whether an open, reproducible pipeline can be extended from broadcast video to tactical outputs, including possession, pass detection, and offside analysis. The system is evaluated both at the component level and as an integrated whole. The degree to which this extension succeeds, and where it encounters limitations, is examined in Chapter 5.

2.3 Object Detection with YOLO

Object detection is the task of simultaneously localising and classifying objects within an image. In broadcast football analysis, the relevant entities are players, goalkeepers, referees, and the ball; for each, a detector must produce a bounding box and a class label. This stage is foundational: missed detections, inaccurate localisation, and class errors propagate directly into tracking, team assignment, homography estimation, and all downstream event reasoning.

Formally, a detector maps an input image I to a set of predictions

$$D = \{(b_i, c_i, s_i)\}_{i=1}^N, \quad (2.1)$$

where b_i denotes a bounding box (centre coordinates, width, and height), c_i a class label, and $s_i \in [0, 1]$ a confidence score reflecting the detector’s certainty that an object of class c_i is present at b_i .

2.3.1 Bounding-Box Localisation and IoU

Localisation quality is measured by the Intersection over Union (IoU) between a predicted box B_p and a ground-truth box B_g :

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \quad (2.2)$$

which takes values in $[0, 1]$, with larger values indicating greater spatial overlap (Figure 2.1). A detection is conventionally counted as a true positive only if its IoU with the nearest unmatched ground-truth box exceeds a threshold and the predicted class is correct [21].

Because a detector typically produces multiple overlapping candidates for the same object, Non-Maximum Suppression (NMS) is applied as a post-processing step. NMS retains the highest-confidence box among strongly overlapping predictions and suppresses the remainder [36].

2.3.2 Precision, Recall, and Mean Average Precision

Detection performance is characterised by the precision–recall trade-off. Precision measures the fraction of predicted boxes that are correct; recall measures the fraction

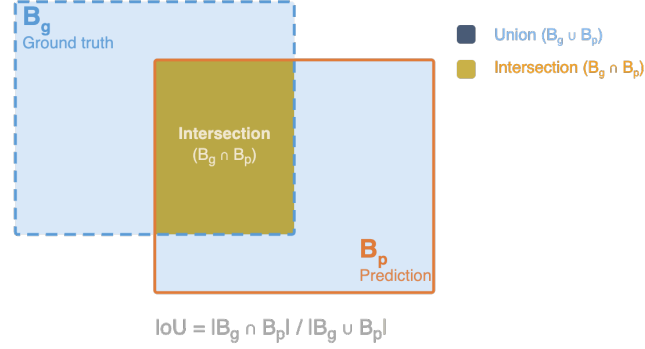


Figure 2.1: Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$.

of ground-truth objects that are found:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (2.3)$$

where TP, FP, and FN denote true positives, false positives, and false negatives respectively. By varying the confidence threshold, a precision–recall curve is obtained; the Average Precision (AP) for a class is the area under this curve, and Mean Average Precision (mAP) is the mean AP across all classes [21].

Two standard operating points are reported in modern benchmarks: mAP_{50} , evaluated at an IoU threshold of 0.50, and mAP_{50-95} , averaged over thresholds from 0.50 to 0.95 in steps of 0.05. The latter is a stricter criterion because it rewards accurate localisation as well as correct classification. In settings where object classes differ substantially in size and detection difficulty, per-class AP values alongside overall mAP give the most informative picture of detector quality.

2.3.3 Detection Paradigms

Modern object detectors broadly fall into two families: two-stage detectors and single-stage detectors. Two-stage methods, exemplified by Faster R-CNN [39], first generate candidate regions via a Region Proposal Network (RPN) and then classify and refine each proposal in a second stage. This separation yields strong accuracy, particularly on small or densely packed objects, but introduces computational overhead that limits

throughput on high-frame-rate video.

Single-stage detectors bypass the proposal step entirely, predicting bounding-box coordinates and class scores directly from the feature map in a single forward pass. Early single-stage architectures such as SSD [25] used pre-defined anchor boxes at multiple spatial scales, requiring careful tuning of anchor shape and aspect-ratio priors for each target domain. More recent architectures adopt an anchor-free formulation, in which each spatial location in the feature map directly regresses the box centre, width, and height without reference to pre-defined templates. This simplifies training, eliminates dataset-specific anchor design, and improves localisation on objects with unusual aspect ratios or very small spatial extent. Table 2.1 summarises the key differences between these paradigms.

Table 2.1: Comparison of object detection paradigms. Single-stage anchor-free detectors simplify training and offer competitive accuracy at high throughput, making them well suited to video-rate applications.

Aspect	Two-stage (e.g. Faster R-CNN)	Single-stage anchor-based (e.g. SSD)	Single-stage anchor-free (e.g. YOLOv8)
Region proposal	Separate RPN stage	None	None
Anchor priors	Yes (at RoI stage)	Yes (predefined)	No
Inference speed	Lower	High	High
Small-object handling	Strong (multi-scale RoI)	Anchor-dependent	Direct regression
Training complexity	Higher	Moderate	Lower

2.3.4 The YOLO Architecture

The YOLO (*You Only Look Once*) family is one of the most widely adopted single-stage detector families. Since its introduction by Redmon et al. [36], YOLO has undergone substantial architectural evolution. The original YOLOv1 divided the input image into a coarse spatial grid and predicted a small number of boxes per cell, achieving real-time speed but suffering limited accuracy on small and overlapping objects. YOLOv2 introduced anchor-based prediction and multi-scale training to improve recall on smaller targets [37]. YOLOv3 added multi-scale feature extraction by predicting at three different resolutions, significantly improving detection across a wider range of object sizes [38]. YOLOv8 [16] represents the current state of the architecture, adopting an anchor-free detection head that directly regresses the box centre,

width, and height from each spatial location in the feature map. This removes the dependency on hand-crafted anchor priors and improves generalisation to objects whose size or aspect ratio falls outside the training distribution.

Feature extraction in modern YOLO variants is performed at multiple spatial scales through a Feature Pyramid Network (FPN) neck [20], which combines high-resolution, semantically weak features from early layers with low-resolution, semantically strong features from later layers. This multi-scale representation is important when objects of very different sizes coexist in the same frame, as is common in broadcast footage where camera zoom produces large scale variation. Figure 2.2 illustrates the general architecture of an anchor-free single-stage detector at an abstract level.

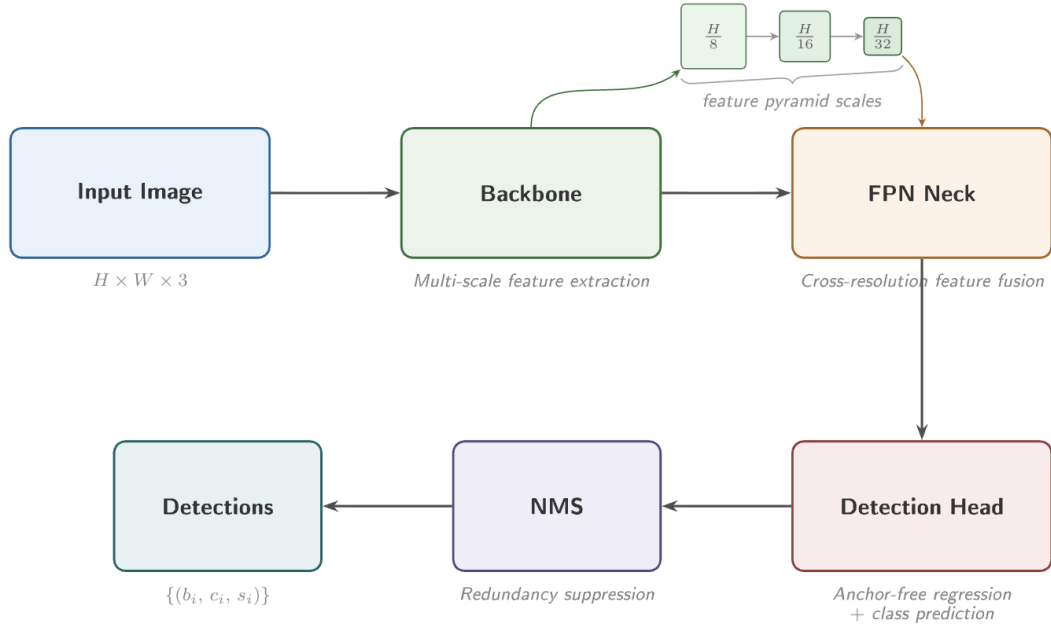


Figure 2.2: Generalised architecture of an anchor-free single-stage object detector. The backbone extracts feature maps at progressively lower spatial resolutions ($H/8$, $H/16$, $H/32$); the FPN neck fuses these across scales; the anchor-free head regresses bounding-box coordinates and class scores at each spatial location; and Non-Maximum Suppression produces the final detection set $\{(b_i, c_i, s_i)\}$ as defined in Equation (2.1).

The confidence score s_i reflects both objectness and class probability. Predictions below a threshold τ are discarded before NMS is applied, introducing a precision–recall trade-off: a high τ reduces false positives at the cost of missed detections, while a low τ recovers more objects at the cost of increased noise. This trade-off is particularly relevant in settings where target objects vary substantially in size and visual salience,

since the optimal operating point may differ across object classes. The implications of per-class threshold selection for small-object detection are discussed in Section 2.4.

Together, the IoU criterion, NMS, precision–recall evaluation, and the anchor-free YOLO formulation provide the detection-theoretic foundation for the implementation decisions described in Chapter 3 and the evaluation metrics reported in Chapter 5.

2.4 Small-Object Detection and Dedicated Ball Detection

Although the ball is the primary object of interest in football, it is also the hardest to detect reliably in broadcast footage. At typical broadcast resolution, the ball subtends only a few dozen pixels in diameter, moves rapidly enough to produce motion blur, and is frequently occluded by players or absent from the frame entirely [13]. These properties place it firmly in the small-object detection regime, where standard detectors trained jointly on all classes tend to underperform because larger, more visually prominent objects dominate the training signal [43].

2.4.1 Why Small Objects Are Harder to Detect

The COCO benchmark defines a small object as one whose bounding box occupies fewer than 32×32 pixels [21]. By this standard, the ball in a wide-angle broadcast frame is consistently small, often below 20×20 pixels depending on camera zoom.

Small-object detection is inherently harder for two reasons. First, the IoU criterion (Equation (2.2)) becomes disproportionately sensitive to localisation error at small scales: a fixed pixel displacement δ produces a much larger IoU drop on a small box than on a large one. As shown in Figure 2.3, a 2-pixel shift reduces IoU from 1.0 to 0.78 on a 16×16 box but only to 0.94 on a 64×64 box. This makes ball detection acutely sensitive to blur, noise, and detector uncertainty. Second, in convolutional architectures the spatial resolution of feature maps decreases with depth; by the deepest layers, small objects may occupy only a single cell or be lost entirely. The FPN neck described in Section 2.3.4 partially mitigates this, but the representation of very small objects remains inherently weaker than that of larger ones [20].

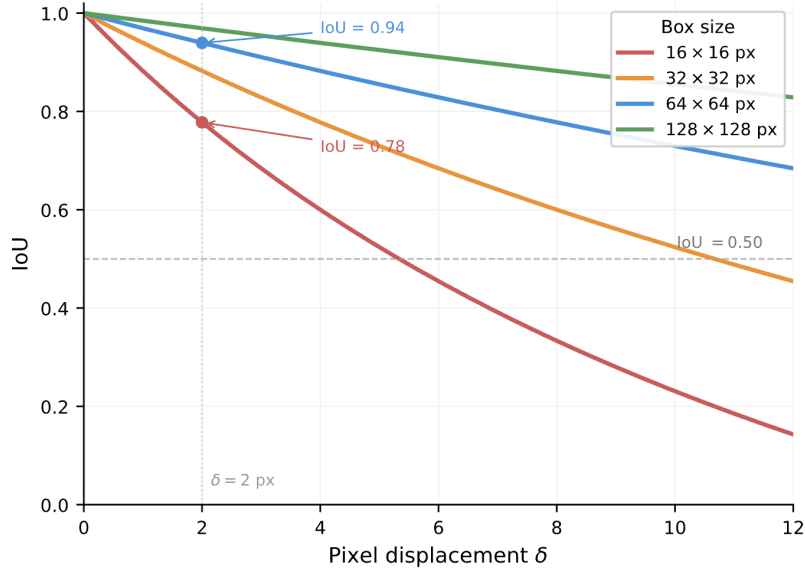


Figure 2.3: Effect of pixel displacement δ on IoU for square bounding boxes of varying size. A single-axis shift is applied to the predicted box while the ground-truth box remains fixed. At $\delta = 2$ pixels, a 16×16 box drops to $\text{IoU} = 0.78$ while a 64×64 box retains $\text{IoU} = 0.94$, illustrating why small-object detection is disproportionately sensitive to localisation error. The dashed line marks the standard mAP_{50} threshold.

2.4.2 Strategies for Small-Object Detection

Several strategies have been proposed to address these difficulties. Multi-scale detection heads, as used in FPN-based architectures, are the most widely adopted architectural response; some methods extend this with additional high-resolution prediction scales or dilated convolutions [20]. At the data level, augmentation techniques such as copy-pasting small object instances into new training locations improve recall on under-represented size categories [18], while inference-time tiling increases effective resolution at the cost of higher computation.

A complementary approach is to train a dedicated detector whose capacity, input resolution, and confidence thresholds are tuned specifically for the small-object class. This avoids the class-imbalance problem inherent in joint training and allows the operating point to be optimised independently. In particular, operating at a low confidence threshold τ maximises recall at the cost of increased false positives — a trade-off that can be managed by downstream temporal filtering rather than within the detector itself [13].

2.4.3 Temporal Filtering of Ball Candidates

When a detector operates at a low confidence threshold, the resulting candidate set at each frame contains both the true ball and spurious detections. The general form of this set at frame t is

$$C_t = \left\{ (\mathbf{b}_t^{(j)}, s_t^{(j)}) \right\}_{j=1}^{N_t}, \quad (2.4)$$

where $\mathbf{b}_t^{(j)}$ is a candidate location and $s_t^{(j)}$ its confidence score.

Temporal coherence provides a principled basis for disambiguation. Because the ball follows a physically continuous trajectory, its inter-frame displacement is bounded even under rapid motion. Two families of filtering exploit this property. Motion-model approaches, such as the Kalman filter described in Section 2.5.1, predict the ball’s next position using an explicit dynamical model and select the closest candidate. These are effective when the assumed model holds, but can diverge under the abrupt accelerations from kicks, deflections, and rebounds that are routine in football. An alternative family enforces only that inter-frame displacement remains below a physically plausible bound, discarding candidates that would imply impossible ball speeds regardless of confidence. This sacrifices predictive power during smooth motion but is more robust to the sharp trajectory changes characteristic of ball dynamics.

The choice between these families, and the specific instantiation used in this project, is discussed in Chapter 3. Chapter 5 reports ball detection performance separately from the general object detector.

2.5 Multi-Object Tracking by Detection

Object detection alone produces independent bounding boxes in each frame, but downstream tasks such as team assignment, heatmap accumulation, speed estimation, and offside reasoning all require object identities to remain consistent over time. Multi-object tracking (MOT) addresses this by associating frame-level detections into persistent trajectories [8].

Modern MOT methods span several paradigms. Graph-based approaches formulate tracking as a network flow or graph optimisation problem over an entire video

sequence, yielding globally consistent trajectories but typically requiring offline processing. Transformer-based methods such as TrackFormer [32] learn associations end-to-end using attention mechanisms, achieving strong performance on standard benchmarks but at higher computational cost. The dominant paradigm for real-time applications remains *tracking-by-detection*, in which a detector first produces candidate boxes and a separate association stage links them across frames. This paradigm is the focus of this section, as it offers the best trade-off between tracking quality and throughput for video-rate processing.

Formally, let

$$D_t = \{d_t^1, d_t^2, \dots, d_t^{n_t}\} \quad (2.5)$$

denote the set of detections at frame t , and let

$$\mathcal{T}_{t-1} = \{\tau_{t-1}^1, \tau_{t-1}^2, \dots, \tau_{t-1}^{m_{t-1}}\} \quad (2.6)$$

denote the set of active tracks from the previous frame. The tracking problem is to assign detections in D_t to tracks in $\mathcal{T}_{t-1}$, while creating new tracks for unmatched detections and terminating or buffering tracks that receive no assignment.

2.5.1 Tracking-by-Detection

Tracking-by-detection methods combine two components: motion prediction, which estimates where each existing track is likely to appear in the current frame, and data association, which matches those predictions to the observed detections.

The standard motion model is the linear Kalman filter [17], which maintains a state vector $\mathbf{x}_t$ comprising the bounding-box geometry (centre, width, height, aspect ratio) and its first-order temporal derivatives. The filter alternates between a prediction step, which propagates the state forward under a constant-velocity assumption,

$$\hat{\mathbf{x}}_t^- = F \mathbf{x}_{t-1}, \quad P_t^- = F P_{t-1} F^\top + Q, \quad (2.7)$$

and an update step, which corrects the prediction using the matched detection $\mathbf{z}_t$,

$$\mathbf{x}_t = \hat{\mathbf{x}}_t^- + K_t (\mathbf{z}_t - H \hat{\mathbf{x}}_t^-), \quad (2.8)$$

where F is the state-transition matrix, P_t^- the predicted error covariance, Q the process noise covariance, H the observation matrix, and K_t the Kalman gain, which balances

confidence in the motion model against confidence in the measurement. The gain is computed from the predicted covariance, observation model, and measurement noise covariance R ; full derivations are given by Kalman [17].

Data association is formulated as a bipartite assignment problem. A cost matrix $C \in \mathbb{R}^{m \times n}$ is constructed where C_{ij} measures the incompatibility between track i and detection j , typically as $1 - \text{IoU}$ using the criterion defined in Equation (2.2). The Hungarian algorithm [19] then finds the minimum-cost one-to-one assignment in $O(n^3)$ time, producing matched pairs, unmatched detections that spawn new tracks, and unmatched tracks that are buffered or deleted.

This paradigm scales efficiently to scenes with many simultaneous objects, but remains sensitive to missed detections, severe occlusion, and visual similarity between targets — all of which are common in broadcast football, where players on the same team share near-identical appearance.

2.5.2 ByteTrack

ByteTrack [46] improves upon the standard tracking-by-detection pipeline by retaining low-confidence detections rather than discarding them. The motivation is that a detector’s confidence score can drop substantially during partial occlusion or motion blur without the underlying object disappearing; discarding such detections prematurely causes unnecessary track fragmentation.

ByteTrack partitions the detector output at frame t into a high-score set

$$D_t^{\text{high}} = \{d \in D_t : s_d \geq \tau_{\text{high}}\} \quad (2.9)$$

and a low-score set $D_t^{\text{low}} = D_t \setminus D_t^{\text{high}}$, where τ_{high} is a confidence threshold. As illustrated in Figure 2.4, association then proceeds in two stages: active tracks are first matched against D_t^{high} using IoU-based cost; tracks that remain unmatched are then compared against D_t^{low} . High-confidence detections establish reliable matches, while low-confidence detections recover continuity for temporarily weakened tracks. Tracks that receive no assignment in either stage are buffered for a fixed number of frames before being deleted.

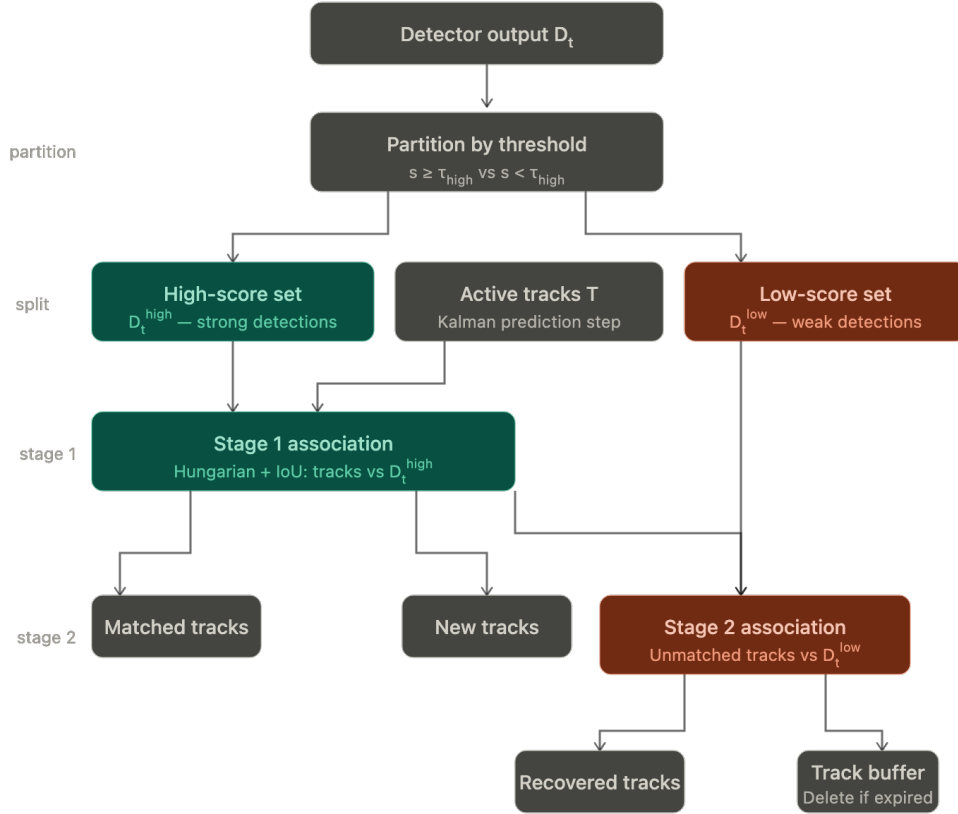


Figure 2.4: ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low} . Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low} . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.

2.5.3 BoT-SORT

BoT-SORT [2] extends the tracking-by-detection framework with two additional mechanisms: camera-motion compensation and appearance-based re-identification. Camera-motion compensation corrects the Kalman filter’s predicted positions for global motion introduced by a moving camera, improving association accuracy during rapid pans or zooms. Appearance-based matching incorporates visual feature similarity as a secondary cost alongside IoU, allowing tracks to be distinguished even when their predicted boxes overlap spatially. The appearance features are derived from a dedicated re-identification model, in contrast to the general-purpose visual embeddings discussed in Section 2.6.

Table 2.2 summarises the key differences between ByteTrack and BoT-SORT. The choice between these trackers for broadcast football analysis is discussed in Chapter 3.

Table 2.2: Comparison of ByteTrack and BoT-SORT. BoT-SORT adds camera-motion compensation and appearance features at the cost of additional computation, while ByteTrack achieves strong performance through its two-stage association strategy alone.

Property	ByteTrack	BoT-SORT
Motion model	Kalman filter	Kalman filter
Camera-motion compensation	No	Yes
Appearance features	No	Re-ID embeddings
Association strategy	Two-stage (high + low)	Two-stage + appearance cost
Computational overhead	Lower	Higher
Robustness to camera motion	Limited	Strong
Robustness to visual ambiguity	IoU-only	IoU + appearance

2.5.4 Tracking Evaluation: HOTA

Tracking quality is assessed using metrics that separately capture localisation accuracy and identity consistency. Multi-Object Tracking Accuracy (MOTA) and Identification F1 (IDF1) are established baselines [8], but both conflate detection and association performance in ways that can obscure the source of errors.

Higher Order Tracking Accuracy (HOTA) was introduced to address this by decomposing overall performance into a detection score and an association score [27]:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha}, \quad (2.10)$$

where DetA_α measures how well predicted boxes overlap ground-truth boxes at localisation threshold α , and AssA_α measures how consistently predicted identities match ground-truth identities across time. The geometric mean ensures that neither component can dominate: a tracker that localises well but fragments identities, or associates well but misses many objects, will score poorly overall. The final HOTA score is averaged over $\alpha \in [0.05, 0.95]$.

2.6 Unsupervised Team Identification from Visual Features

In broadcast football footage, team identity is not available as an explicit detector output. Players from the two outfield teams must therefore be grouped using visual cues, without labelled examples for the specific match being analysed. This makes team identification an unsupervised clustering problem rather than a supervised classification task. A useful representation must distinguish jerseys reliably while remaining robust to pose variation, background clutter, scale changes, and illumination differences.

2.6.1 Approaches to Team Identification

Existing methods for unsupervised team identification differ primarily in how they represent player appearance before clustering. Three broad families can be distinguished.

Colour-histogram features. The simplest approach extracts colour histograms from player crops, typically in HSV or Lab colour space, and clusters the resulting vectors [15]. Histograms are fast to compute and effective when the two teams’ dominant jersey colours are clearly distinct. However, they discard spatial structure entirely and are sensitive to illumination variation, background contamination from the pitch, and kits whose colour distributions overlap (e.g. white versus light grey).

Supervised CNN features. Convolutional neural networks trained on person re-identification datasets produce embeddings that capture richer visual structure than raw colour statistics. These features encode texture, pattern, and shape alongside colour, making them more robust to illumination and pose variation. However, re-identification models are typically trained to distinguish *individuals*, whereas team

identification requires distinguishing *groups* that share a common appearance. This mismatch can reduce clustering separability when intra-team variation (e.g. from different player builds or kit numbers) is large relative to inter-team differences.

Vision–language model embeddings. Vision–language models (VLMs) trained with contrastive objectives learn to align image and text representations over very large and diverse datasets. This training encourages the encoder to organise its feature space along visually and semantically meaningful axes — including colour, texture, and pattern — without requiring domain-specific supervision. Models in this family include CLIP [35], SigLIP [45], and DINOv2 [34]. Their embeddings tend to be more robust than both hand-crafted colour features and re-identification features for team-level grouping, because visually salient properties such as jersey colour and kit design correspond naturally to the axes along which contrastive training organises the feature space.

Table 2.3 summarises the trade-offs between these three representation families.

Table 2.3: Comparison of representation strategies for unsupervised team identification. Vision–language model embeddings offer the best trade-off between robustness and generality, at the cost of higher-dimensional feature vectors.

Property	Colour histograms	Re-ID CNN	VLM embeddings
Captures colour	Yes	Yes	Yes
Captures texture/pattern	No	Partially	Yes
Illumination robustness	Low	Moderate	High
Domain supervision required	None	Re-ID dataset	None
Feature dimensionality	Low	Medium	High
Sensitivity to crop region	High	Moderate	Moderate

2.6.2 Deep Visual Embeddings

Regardless of the specific model used, the embedding step maps each player crop to a feature vector in a high-dimensional space. Formally, let I_i denote the image crop of player i . A pretrained visual encoder defines a mapping

$$f : I_i \mapsto \mathbf{z}_i \in \mathbb{R}^d, \quad (2.11)$$

where $\mathbf{z}_i$ captures semantically useful visual information. The aim is that players from the same team produce embeddings that lie closer together in this space than those from opposing teams.

The quality of the resulting embedding depends not only on the encoder but also on the input region. In player detection crops, the jersey torso typically contains the most discriminative signal for team identity, while the lower body, head, and any visible background contribute noise. Cropping strategies that isolate the most informative region before encoding can therefore improve cluster separability — a consideration addressed in the implementation described in Chapter 3.

2.6.3 Dimensionality Reduction and Clustering

VLM embeddings are typically high-dimensional (often $d > 500$). Clustering directly in such spaces can be unstable because Euclidean distances become less discriminative as dimensionality increases — a phenomenon known as the curse of dimensionality [1]. A standard response is to apply dimensionality reduction before clustering, producing a compact representation that preserves useful structure.

Let $\mathbf{z}_i \in \mathbb{R}^d$ be the embedding of player i and let

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, \quad d' \ll d, \quad (2.12)$$

be a dimensionality-reduction mapping, yielding the reduced representation $\mathbf{y}_i = \phi(\mathbf{z}_i)$. Two widely used methods are Principal Component Analysis (PCA), which projects onto the directions of maximum global variance, and Uniform Manifold Approximation and Projection (UMAP) [31], which preserves local neighbourhood structure. The choice between these depends on whether the downstream clustering task is better served by global variance preservation or local similarity preservation. For team identification, where the relevant structure is similarity between same-kit players rather than global variance across all crops, neighbourhood-preserving methods are generally better aligned [31].

Given the reduced representations $\{\mathbf{y}_i\}_{i=1}^N$, k -means clustering [28] partitions them into k groups by minimising the within-cluster sum of squares:

$$\min_{\{\mu_j\}_{j=1}^k} \sum_{i=1}^N \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2, \quad (2.13)$$

where μ_j is the centroid of cluster j . Setting $k = 2$ separates the two outfield teams

without requiring ground-truth labels.

2.6.4 Temporal Stability and Special Cases

Per-frame clustering predictions are often noisy: a player may be assigned inconsistently across neighbouring frames because of blur, partial occlusion, or crop quality variation. Since a player’s team identity is constant over the duration of a track, temporal smoothing of the predicted labels is a natural post-processing step. The general principle is to aggregate predictions over a sliding window and assign the most frequently predicted label, reducing short-term jitter while preserving responsiveness to genuine corrections.

A further challenge arises with goalkeepers, who typically wear a kit that differs from both outfield teams. This makes appearance-based clustering unreliable for goalkeepers regardless of the embedding model used. Alternative assignment strategies — for example, exploiting spatial context rather than appearance — are therefore required for this class. The specific approaches to temporal smoothing and goalkeeper resolution adopted in this project are described in Chapter 3.

2.7 Pitch Keypoint Detection and Planar Homography

Many outputs of a broadcast football pipeline — bird’s-eye projection, heatmaps, speed estimation, possession analysis, and offside reasoning — require positions to be expressed in a common pitch coordinate system rather than in image pixels. This is achieved by estimating a geometric mapping from the image plane to a canonical pitch model, a process known as field registration [43, 42].

2.7.1 Registration Approaches

A football pitch contains a fixed set of semantic landmarks — line intersections, penalty-box corners, and circle tangent points — whose locations are precisely known in a standard top-down representation. Several families of methods exploit this structure for registration.

Keypoint-based methods. These detect individual pitch landmarks in the broadcast frame, establishing point correspondences between image space and pitch space that are used to estimate a geometric transformation. Keypoint detection can be performed

using pose-estimation architectures, which regress the 2D image coordinates and visibility confidence of a predefined set of landmarks [16]. The main requirement is that the detected points are spatially distributed across the frame, since correspondences clustered in one region poorly constrain the transformation.

Line and edge-based methods. Rather than detecting discrete points, these methods detect pitch lines or edges and align them with a canonical pitch template. Magera et al. [29] proposed BroadTrack, which uses line-segment matching to achieve strong calibration accuracy on the SoccerNet benchmark. Line-based methods can be more robust than keypoint methods when individual landmarks are ambiguous, but require reliable line detection under varying broadcast conditions.

Direct regression methods. An alternative family bypasses explicit landmark detection entirely and instead regresses the parameters of the image-to-pitch transformation directly from the input frame using a neural network [33]. These methods avoid the need for keypoint or line detection but are less interpretable and can struggle to generalise across stadiums with different pitch markings or camera configurations.

The choice among these approaches involves trade-offs between robustness, interpretability, and generalisation. The specific method used in this project is described in Chapter 3.

2.7.2 Planar Homography

Regardless of the detection method, the geometric mapping between image coordinates and pitch coordinates is modelled by a planar homography, justified by the assumption that the football pitch is approximately planar. A homography is a projective transformation represented by a 3×3 matrix H , defined up to a non-zero scalar factor since H and λH define the same transformation for any $\lambda \neq 0$. Using homogeneous coordinates,

$$\tilde{\mathbf{X}} \sim H\tilde{\mathbf{x}}, \quad (2.14)$$

where $\tilde{\mathbf{x}} = (u, v, 1)^\top$ and $\tilde{\mathbf{X}} = (x, y, 1)^\top$ denote image and pitch points in homogeneous form respectively.

Formally, let $(\mathbf{x}_i, \mathbf{X}_i)$ denote the i -th point correspondence, where $\mathbf{x}_i = (u_i, v_i)^\top$ is the image-space location of landmark i and $\mathbf{X}_i = (x_i, y_i)^\top$ is its known location on the

canonical pitch. Since H has eight degrees of freedom up to scale, at least four non-collinear correspondences are required to estimate it. With $N > 4$ correspondences the system is overdetermined and is solved in a least-squares sense using the Direct Linear Transform (DLT) [14]. The DLT constructs a linear system $A\mathbf{h} = \mathbf{0}$ from the correspondences, where $\mathbf{h} \in \mathbb{R}^9$ is the vectorised form of H . The solution minimises $\|A\mathbf{h}\|_2$ subject to $\|\mathbf{h}\|_2 = 1$ and is obtained in closed form as the right singular vector of A corresponding to its smallest singular value.

2.7.3 Outlier Rejection with RANSAC

The DLT solution assumes that all correspondences are correct. In practice, keypoint detectors may produce misidentified or poorly localised landmarks, and even a single incorrect correspondence can severely distort the estimated homography. Random Sample Consensus (RANSAC) [11] addresses this by repeatedly fitting a homography to a minimal random subset of four correspondences and scoring each candidate against the remaining points. The candidate with the largest consensus set — the set of correspondences whose reprojection error falls below a tolerance — is selected, and the final homography is re-estimated from the full consensus set using DLT. This makes the estimation robust to a moderate proportion of outlier correspondences, which is important in broadcast settings where landmark detection quality varies with camera angle, zoom, and occlusion by players or graphics overlays.

2.7.4 Reprojection Error and Bird’s-Eye Mapping

Once H has been estimated, any point assumed to lie on the pitch plane can be mapped from image space to pitch space, yielding a bird’s-eye representation in which positions can be compared over time without perspective distortion [43].

Registration quality is quantified by the reprojection error. For a correspondence $(\mathbf{x}_i, \mathbf{X}_i)$, this is

$$e_i = \|\mathbf{X}_i - \pi(H\tilde{\mathbf{x}}_i)\|_2, \quad (2.15)$$

where $\pi(\cdot)$ converts homogeneous coordinates to Cartesian coordinates. The mean reprojection error over all N correspondences,

$$\bar{e} = \frac{1}{N} \sum_{i=1}^N e_i, \quad (2.16)$$

measures how well the estimated transformation aligns the detected landmarks with their canonical positions. Low mean reprojection error is necessary but not sufficient: an accurate fit to the detected correspondences does not guarantee that the homography generalises well to other points on the pitch, particularly if the correspondences are spatially clustered.

Since homography error propagates to all downstream spatial analysis — shifting projected player positions, distorting heatmaps, and potentially altering offside decisions — accurate field registration is a geometric prerequisite for the entire pipeline rather than an isolated preprocessing step [42].

2.8 Spatial Analytics in Pitch Space

Once detections have been registered to a canonical pitch, spatial analysis can be performed in a coordinate system that is independent of camera perspective. Many tactical quantities — player positioning, spatial activity, speed, and distance covered — are meaningful only when expressed on the pitch plane rather than in image pixels [43, 12].

2.8.1 Player Projection onto the Pitch

To analyse player motion on the playing surface, each detection must be represented by a single point on the pitch plane. The standard convention is to use the bottom-centre of the bounding box as the ground-contact proxy, since this best approximates the location at which the player contacts the pitch surface. Using the box centre would yield a point above the ground plane, producing a systematic projection error that varies with player height and distance from the camera.

Let $B_t^{(i)}$ denote the bounding box of player i at frame t , and let $\mathbf{q}_t^{(i)} \in \mathbb{R}^2$ denote its bottom-centre anchor in image coordinates. Given the image-to-pitch homography H , the projected pitch-space position is

$$\mathbf{p}_t^{(i)} = \pi\left(H\tilde{\mathbf{q}}_t^{(i)}\right), \quad (2.17)$$

where $\tilde{\mathbf{q}}_t^{(i)}$ is the homogeneous form of the anchor point and $\pi(\cdot)$ converts homogeneous coordinates to Cartesian coordinates. Once each player is represented by a pitch-space trajectory $\{\mathbf{p}_t^{(i)}\}_{t=1}^T$, positions can be compared across frames in a common geometric reference and aggregate statistics derived over time.

2.8.2 Heatmap Construction

A player heatmap summarises spatial activity over a sequence by estimating the density of projected positions on the pitch. The pitch is divided into a discrete $M \times N$ grid and an occupancy count is accumulated in each cell:

$$H_i(u, v) = \sum_{t=1}^T \mathbf{1} \left[(u, v) = \text{bin}(\mathbf{p}_t^{(i)}) \right], \quad (2.18)$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\text{bin}(\cdot)$ assigns a pitch position to its grid cell. Because raw counts are sparse, the histogram is smoothed by convolution with a Gaussian kernel G_σ :

$$\tilde{H}_i = H_i * G_\sigma, \quad (2.19)$$

where σ controls the spatial scale of smoothing. The result is a continuous-looking density map useful for inspecting player roles, movement concentration, and broad positional behaviour. Heatmap quality depends directly on the reliability of the projected trajectories: registration errors shift the density spatially, while tracking fragmentation can split a single player's activity across multiple incomplete maps.

2.8.3 Speed and Distance Estimation

Once player positions are available in pitch coordinates, movement statistics are estimated from temporal differences. Let f_s denote the video frame rate in Hz, so that the inter-frame time interval is $\Delta t = 1/f_s$. A finite-difference estimate of velocity for player i at frame t is

$$\mathbf{v}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)}}{\Delta t}, \quad (2.20)$$

with instantaneous speed

$$s_t^{(i)} = \left\| \mathbf{v}_t^{(i)} \right\|_2. \quad (2.21)$$

Total distance covered is approximated by summing successive displacements:

$$d^{(i)} = \sum_{t=2}^T \left\| \mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)} \right\|_2. \quad (2.22)$$

These estimates are sensitive to noise in the projected positions. Small localisation errors are amplified by finite differencing, producing spurious speed spikes, and

identity switches in the tracker introduce artificial displacement jumps. Two complementary mitigations are standard in the literature. First, trajectory smoothing — using a moving average, Savitzky-Golay filter, or similar — attenuates high-frequency noise before differentiation. Second, a plausibility cap discards speed estimates exceeding a physically reasonable upper bound for human running speed, acting as a hard gate analogous to the displacement constraint in ball detection (Section 2.4). Only displacements corresponding to speeds below the cap contribute to the distance accumulation in Equation (2.22).

These caveats are well established in the sports-performance literature, where the accuracy of running statistics depends strongly on how the underlying trajectories are obtained and filtered [3, 23]. In broadcast-derived pipelines, where positional accuracy is lower than in dedicated tracking installations, careful treatment of noise is especially important.

The spatial analytics described in this section are evaluated in Chapter 5, where their dependence on registration and tracking quality is examined directly.

2.9 Event Reasoning and Match Statistics

Once players and the ball have been projected into pitch space, higher-level match events can be expressed as rules over trajectories rather than as isolated image observations. Three event-level outputs are particularly relevant to broadcast football analysis: possession, passes, and offside. These quantities are not observed directly by the detector; instead, they are inferred from tracked spatial relationships over time, making them dependent on both geometric registration and temporal consistency [43].

2.9.1 Ball Possession Estimation

Possession estimation aims to determine which team controls the ball at each moment. Several definitions exist in the literature, differing in complexity and in the information they require.

Proximity-based possession. The simplest operational definition assigns the ball to the nearest eligible player. Let $\mathbf{b}_t \in \mathbb{R}^2$ denote the ball position in pitch space at frame t , and let $\mathbf{p}_t^{(i)} \in \mathbb{R}^2$ denote the position of player i , drawn from a candidate set $\mathcal{A}_t$ of

on-pitch players excluding referees. The ball holder at frame t is defined as

$$h_t = \arg \min_{i \in \mathcal{A}_t} \left\| \mathbf{b}_t - \mathbf{p}_t^{(i)} \right\|_2, \quad (2.23)$$

subject to a maximum possession radius $d_{\max}$: if no player falls within this distance, possession is unassigned. This definition is interpretable and naturally expressed in pitch coordinates, but it is a heuristic approximation that can fail when the ball is airborne, when multiple players contest simultaneously, or when localisation error changes which player appears closest.

Pitch-control models. More sophisticated approaches model possession through spatial influence rather than simple proximity. Fernandez and Bornn [10] proposed a pitch-control framework in which each player’s influence is modelled as a function of position, velocity, and orientation, producing a continuous field over the pitch surface. Possession is then defined by which team’s influence dominates at the ball’s location. These models capture tactical nuance that proximity alone cannot, but they require richer input (player velocities and orientations) and are more sensitive to tracking noise.

Temporal stabilisation. Regardless of the underlying definition, raw per-frame possession estimates are noisy. A player may flicker in and out of possession due to detector jitter or brief localisation errors. Temporal smoothing — requiring that possession be sustained for a minimum duration before being confirmed — is a standard post-processing step that reduces spurious transitions [12].

2.9.2 Pass Detection

A pass can be modelled as a stable transition in ball possession from one player to another. Given a possession sequence $\{h_t\}$, a candidate pass event is detected when the assigned ball holder changes identity, subject to stability conditions that filter out spurious transitions caused by brief detection noise [12].

The moment at which possession changes provides a practical approximation to the instant at which the ball is played. This temporal anchor is central to offside analysis, making pass detection not merely a statistic in itself but a prerequisite for the event reasoning described in the following subsection. The specific stability conditions and filtering logic used in this project are described in Chapter 3.

2.9.3 Offside Rule Formalisation

Offside is one of the most spatially structured rules in football and a natural target for pitch-space reasoning. Under Law 11 of the Laws of the Game, a player is in an offside position if, at the moment the ball is played by a team-mate, they are in the opponents' half and nearer to the opponents' goal line than both the ball and the second-last opponent [41].

For an automated system, a tractable formulation is to detect *positional offside*: whether an attacking player satisfies the geometric conditions at the relevant moment. Let $x_t^{(a)}$ denote the pitch-space coordinate of an attacking player along the attacking axis, $x_t^{(b)}$ the corresponding ball coordinate, and $x_t^{(o_2)}$ the coordinate of the second-last opponent. The positional offside condition is

$$x_t^{(a)} > \max(x_t^{(b)}, x_t^{(o_2)}), \quad (2.24)$$

together with the requirement that the attacker is in the opponents' half. The max operator captures that the attacker must be beyond both the ball and the second-last opponent simultaneously.

This formulation highlights why pitch-space geometry is essential: offside cannot be judged reliably in image space because perspective distortion alters apparent distances and orderings along the goal line. By mapping all relevant entities into a common pitch coordinate system, the comparison reduces to a one-dimensional ordering problem along the attacking direction.

Attacking direction. Evaluating Equation (2.24) requires knowing which direction each team is attacking. In dedicated tracking systems this information is provided as metadata, but in a broadcast pipeline it must be inferred from the data. This inference introduces an additional dependency on the stability of team identities and geometric calibration: an incorrect attacking direction would invert the offside comparison entirely. The specific method used to determine attacking direction in this project is described in Chapter 3.

Limitations. The formulation in Equation (2.24) is a deliberate simplification. Law 11 depends on the precise moment the ball is played — which must be approximated from the possession sequence — and on whether an offside-positioned player becomes actively involved in play, a judgement that a trajectory-based pipeline cannot recover.

Automated offside analysis should therefore be interpreted as positional offside detection rather than a complete implementation of the law [41]. The accuracy of this approximation is examined in Chapter 5.

2.10 Error Propagation in a Multi-Stage Pipeline

A broadcast-football analysis system is naturally a multi-stage pipeline in which later outputs are produced not directly from the input video but from intermediate estimates generated by earlier modules. Errors therefore do not remain local to the component in which they arise; they propagate forward and affect all downstream analysis. This is an important theoretical consideration in end-to-end sports-video pipelines, where detection, tracking, registration, and event reasoning are tightly coupled [43, 42].

At an abstract level, the pipeline can be viewed as a composition of functions

$$y = f_n(f_{n-1}(\cdots f_2(f_1(x)) \cdots)), \quad (2.25)$$

where x is the input video, each f_k represents one processing stage, and y is a final output such as a heatmap, possession estimate, or offside decision. In a broadcast football pipeline, the stages $f_1, \dots, f_n$ typically correspond to object detection, multi-object tracking, team identification, pitch registration, and pitch-space event reasoning. Each stage consumes the structured output of its predecessors rather than the raw video. If the output of stage k is perturbed by some error δ_k , the downstream effect on the final output,

$$\Delta y = f_n(\cdots f_{k+1}(f_k(x_k) + \delta_k) \cdots) - y, \quad (2.26)$$

depends both on the magnitude of δ_k and on the sensitivity of all subsequent stages to it.

Figure 2.5 illustrates the dependency structure and the principal error propagation paths in a broadcast football pipeline. Several forms of propagation are especially consequential. A missed detection may break a player trajectory, which can in turn reduce the stability of team assignment and corrupt movement statistics. An identity switch in the tracker may artificially inflate estimated distance covered or create false changes in possession. A registration error in the homography may shift projected player positions, distorting heatmaps and changing the relative ordering of attackers and defenders in offside analysis. An incorrect team label may cause possession, pass counts, or attacking direction to be inferred incorrectly, even if the underlying detections and tracks

are geometrically accurate.

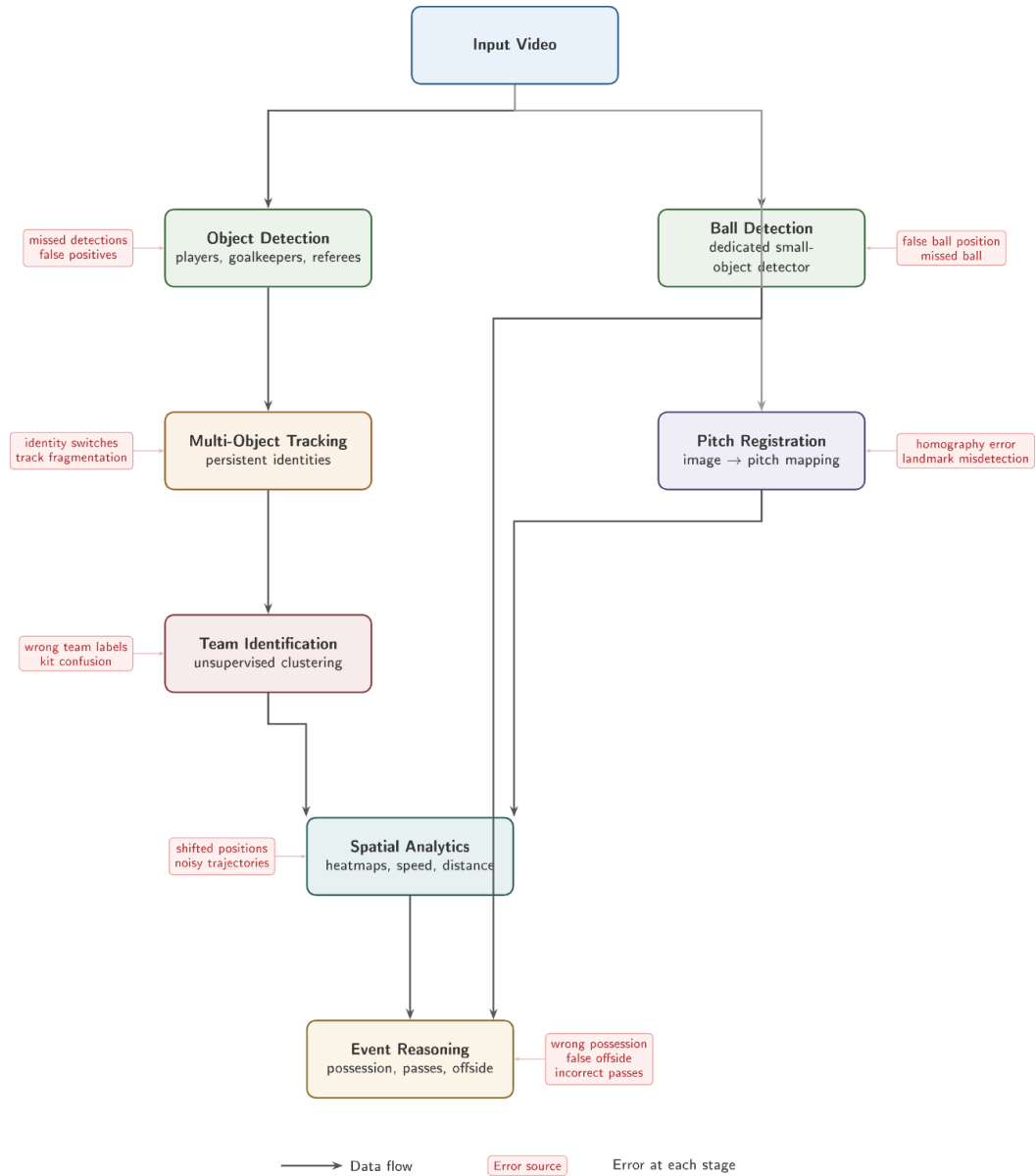


Figure 2.5: Error propagation paths in a broadcast football analysis pipeline. Solid arrows show the primary data flow between processing stages; dashed arrows indicate how errors at each stage propagate to downstream outputs. A detection error (e.g. a missed player) can cascade through tracking, team identification, and spatial analytics, ultimately affecting event-level outputs such as possession and offside decisions.

This dependency structure means that downstream outputs cannot be interpreted independently of upstream quality. A poor offside decision may reflect not only a

weakness in the event logic itself, but also earlier errors in ball localisation, player tracking, team assignment, or pitch registration. Similarly, noisy speed estimates may originate from homography jitter or identity fragmentation rather than from the estimation formula alone.

End-to-end evaluation of a broadcast football pipeline should therefore combine component-level metrics with explicit analysis of downstream behaviour [12, 27]. Component-level evaluation identifies where errors originate; system-level evaluation shows how those errors affect the usefulness of the integrated outputs. This motivates the evaluation strategy adopted in Chapter 5, where both individual modules and the final pipeline outputs are assessed.

2.11 Chapter Summary

This chapter has established the theoretical foundation for each component of a broadcast football analysis pipeline. The progression from object detection and small-object detection through multi-object tracking, unsupervised team identification, pitch registration, spatial analytics, and event reasoning reflects a deliberate decomposition: each stage reduces a complex vision problem to a well-defined input–output contract, allowing errors to be localised and components to be evaluated independently. The error propagation analysis in Section 2.10 makes explicit that this modularity does not eliminate inter-stage dependencies; it only makes them tractable.

For each stage, the chapter has surveyed the relevant approaches — contrasting detection paradigms, tracking strategies, team-identification representations, and registration methods — and introduced the metrics and formalisms needed to evaluate them, including IoU, mAP, HOTA, DLT homography, reprojection error, and the positional offside condition. These concepts are used directly in Chapter 3 to motivate implementation decisions and in Chapter 5 to assess pipeline performance at both the component and system level.

Chapter 3

Methodology and Implementation

3.1 Chapter Overview

This chapter describes the methodology used to design, implement, and evaluate the proposed broadcast football video analysis system. Building on the background and related work discussed in Chapter 2, the aim is to construct an open and reproducible pipeline that takes a single broadcast football video as input and produces both visual and quantitative tactical outputs.

The chapter begins by defining the requirements and design objectives that guided the system. It then presents the overall system architecture, including the main processing stages, intermediate representations, and final outputs. The following sections describe the datasets and training procedures used for the detection and pitch keypoint models, before explaining the main pipeline components: object and ball detection, multi-object tracking, team identification, pitch registration, and tactical output generation.

Particular attention is given to the design choices made at each stage, since the performance of later components depends directly on the reliability of earlier ones. For example, ball detection affects possession and pass estimation, while pitch registration affects heatmaps, speed calculation, and offside candidate detection. The chapter therefore describes not only what was implemented, but also why each method was selected and how errors are managed where possible.

The final sections describe the evaluation methodology and reproducibility measures. These include component-level evaluation, end-to-end pipeline evaluation, ablation studies, runtime measurement, and error analysis. Results from these evaluations are reported in Chapters 4 and 5.

3.2 Requirements and Design Objectives

The methodology of the system was guided by the overall aim of producing an open and reproducible pipeline for tactical football analysis from broadcast video alone. This required the system to recover player, ball, team, and pitch information from a single moving-camera feed, and to transform these intermediate outputs into analytically useful representations such as heatmaps, possession statistics, pass events, and candidate offside situations.

To ensure that the design remained aligned with this aim, the system was specified in terms of functional requirements, non-functional requirements, and success criteria. These requirements were used both to guide implementation choices and to define the basis for later evaluation.

3.2.1 Functional Requirements

The functional requirements describe the core capabilities that the system was expected to provide.

- **FR1: Object detection.** The system must detect the main entities required for football analysis, namely players, goalkeepers, referees, and the ball, from broadcast frames.
- **FR2: Temporal tracking.** The system must maintain player identities across frames so that movement-based analytics can be computed over time.
- **FR3: Team identification.** The system must assign players to teams without requiring match-specific labelled training data for each new fixture.
- **FR4: Pitch registration.** The system must estimate a mapping from image coordinates to a canonical pitch model so that image-space detections can be analysed in real-world pitch space.
- **FR5: Tactical output generation.** The system must derive higher-level outputs from the tracked and registered data, including player heatmaps, speed and distance estimates, team possession statistics, pass events, and candidate offside situations.

- **FR6: Visual output generation.** The system must produce interpretable outputs that support inspection of intermediate and final results, including annotated broadcast video, bird’s-eye visualisations, side-by-side renderings, JSON statistics, and heatmap images.

3.2.2 Non-functional Requirements

The non-functional requirements describe the practical and technical qualities expected of the system beyond its core functionality.

- **NFR1: Reproducibility.** The system should be executable from stored model weights, configuration parameters, and documented runtime dependencies, so that results can be reproduced consistently.
- **NFR2: Modularity.** The pipeline should be structured as a set of separable stages so that individual components, such as the tracker, team classifier, or pitch registration module, can be analysed, replaced, or improved independently.
- **NFR3: Robustness to broadcast conditions.** The system should remain usable under common broadcast challenges, including camera motion, partial occlusion, missed detections, and noisy pitch registration.
- **NFR4: Interpretability.** Intermediate and final outputs should be inspectable by a human observer, allowing incorrect detections, unstable tracks, registration failures, and tactical inference errors to be identified and analysed.
- **NFR5: Practical runtime.** The system should process complete match clips in a practically manageable time, even if the design prioritises analytical accuracy over real-time deployment.

3.2.3 Success Criteria

The success of the system was defined in terms of both component-level and end-to-end behaviour.

At the component level, the detector should localise the relevant classes with useful accuracy, the tracker should preserve identities with limited fragmentation, the team identification stage should assign stable team labels, and the pitch registration

stage should produce sufficiently consistent transformations for downstream pitch-space analysis. At the end-to-end level, the pipeline should generate coherent visualisations and plausible tactical outputs across real broadcast clips.

In practical terms, the project was considered successful if it could:

- produce stable annotated video outputs showing tracked and labelled entities;
- generate bird’s-eye projections that remain visually aligned with the broadcast scene;
- produce usable player heatmaps and per-player spatial statistics;
- estimate possession and pass events in a way that is consistent with manual inspection of representative clips; and
- identify candidate offside situations in pitch space under the assumptions defined later in this chapter.

These success criteria also shaped the evaluation design. Rather than judging the project only by model-level training metrics, the system is assessed both at the level of individual components and as an integrated pipeline, since the ultimate objective is not only accurate reconstruction, but analytically useful football outputs from broadcast video.

3.3 System Architecture

The system was designed as a modular pipeline in which each stage transforms the output of the previous stage into a representation that is more useful for football analysis. This design was chosen because the overall task is too complex to solve reliably as a single monolithic prediction problem: player locations, ball position, team identity, pitch geometry, and tactical events each require different forms of visual and geometric reasoning. A modular architecture also supports debugging, replacement of individual components, and component-level evaluation, which are important for reproducibility and critical analysis.

3.3.1 Pipeline Overview

Figure 3.1 shows the overall architecture of the pipeline. The input is a broadcast football video. The output is an annotated video together with optional bird’s-eye visualisations, side-by-side videos, heatmaps, possession and pass statistics, and detected offside candidate events.

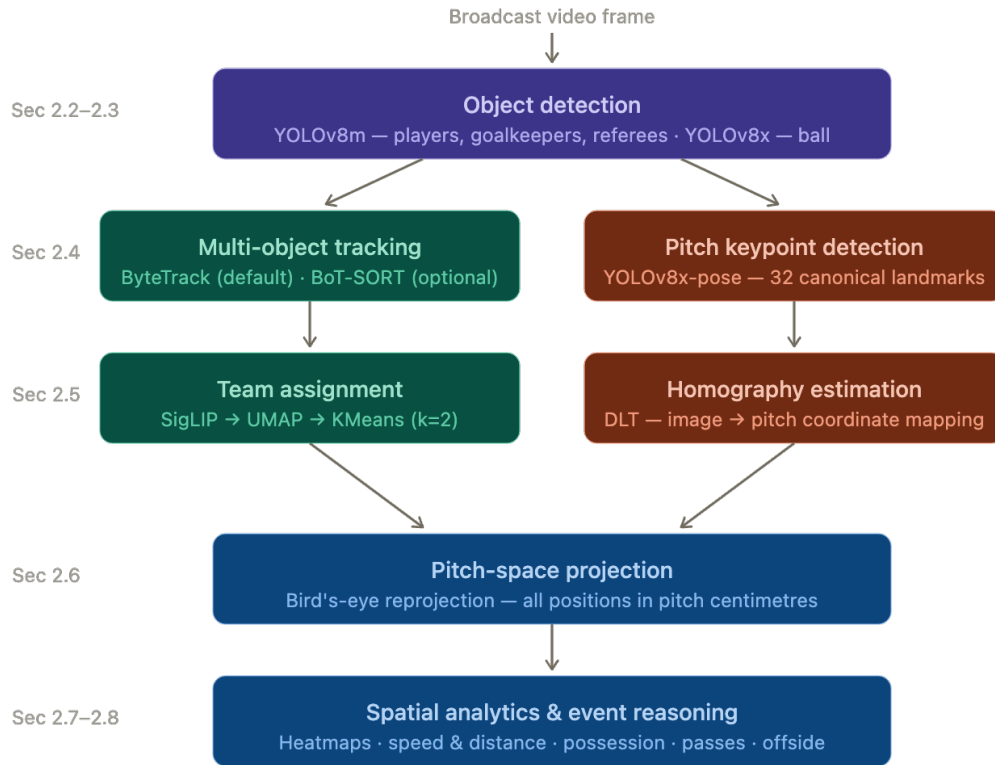


Figure 3.1: Overview of the proposed broadcast football video analysis pipeline. The system begins with object and ball detection, then applies tracking, team identification, pitch registration, and tactical analysis to produce visual and quantitative outputs.

The pipeline consists of two main phases. The first is a warm-up phase used to fit the team identification model from player crops sampled near the beginning of the video. The second is a per-frame processing phase in which detections, tracks, team labels, pitch coordinates, visualisations, and statistics are updated continuously.

3.3.2 Data Flow and Intermediate Representations

Each frame is first processed by the main object detector, which identifies players, goalkeepers, referees, and ball candidates. A dedicated ball detector is then applied to improve ball localisation, since the ball is small and frequently missed by the general detector. The resulting detections are passed to a multi-object tracker, which assigns persistent track identifiers to players across frames.

Team identification is performed using player crops. During the warm-up phase, player crops are sampled from the video and used to fit an unsupervised team classifier. During the per-frame phase, each tracked player is assigned a team label, and temporal smoothing is applied to reduce frame-level instability. Goalkeepers are handled separately using spatial proximity to the detected outfield teams, while referees are assigned to a non-team class.

Pitch registration is performed by detecting pitch keypoints and estimating a homography between the broadcast frame and a canonical pitch model. This allows image-space detections to be transformed into pitch-space coordinates. Pitch-space coordinates are then used to generate bird's-eye visualisations, heatmaps, speed and distance estimates, possession statistics, pass events, and offside candidate detections.

The main intermediate representations used by the system are:

- **Bounding boxes**, representing detected players, goalkeepers, referees, and ball candidates in image space.
- **Track identifiers**, representing temporally consistent player identities across frames.
- **Team labels**, representing the inferred team membership of each tracked player.
- **Pitch keypoints**, representing detected landmarks on the football pitch.
- **Homography matrices**, representing the mapping from image coordinates to pitch coordinates.
- **Pitch-space coordinates**, representing object locations in a canonical coordinate system suitable for tactical analysis.

3.3.3 System Outputs

The system produces several forms of output so that both intermediate behaviour and final tactical results can be inspected. The main output is an annotated broadcast video

in which detected players, goalkeepers, referees, and the ball are visualised with track and team information. When pitch registration is enabled, the system can also generate a bird’s-eye video and a side-by-side video combining the original broadcast view with the pitch-space representation.

In addition to video outputs, the pipeline can export structured data. Player and team statistics are written to JSON files, including possession estimates, pass counts, player speed estimates, and distance travelled. Candidate offside events are also exported as structured records, allowing them to be inspected after processing. Player heatmaps are saved as image files and raw numerical arrays so that the spatial distribution of each tracked player can be analysed or reproduced later.

This separation between visual outputs and structured outputs is important because the system must support both qualitative inspection and quantitative evaluation. Visual outputs make it possible to diagnose errors such as missed detections, team-label switches, or unstable registration, while structured outputs allow the downstream tactical analysis to be evaluated more systematically in Chapters 4 and 5.

3.4 Datasets and Model Training

Three supervised models were trained for the system: a main object detection model, a dedicated ball detection model, and a pitch keypoint detection model. The main detector provides the player, goalkeeper, referee, and initial ball detections used by the tracking pipeline. The dedicated ball detector is used to improve ball localisation, while the pitch keypoint model provides the landmarks required for homography estimation. Team identification does not use a separately trained supervised model; instead, it is fitted at runtime using player crops from the input video.

3.4.1 Object Detection Dataset and Training

The main object detector was trained on the Roboflow *football-updated-2024* dataset, version 2. This dataset contains four classes: *ball*, *goalkeeper*, *player*, and *referee*. The training split contained 618 images and the validation split contained 75 images.

A YOLOv8m object detection model was fine-tuned on this dataset for 50 epochs. The model was trained with an image size of 1280 pixels and a batch size of 4. This model size was selected as a compromise between detection accuracy and runtime cost,

since the detector is executed throughout the full video processing pipeline. The resulting model weights were saved as `models/object_detection/best.pt` and used as the main detector during pipeline inference.

3.4.2 Ball Detection Dataset and Training

A separate ball detector was trained because the ball is substantially smaller than the other object classes and is often affected by motion blur, occlusion, or compression artefacts. In preliminary pipeline testing, relying only on the ball class from the main detector produced missed detections that affected possession, pass detection, and off-side candidate generation. A dedicated model therefore allowed the detection stage to be specialised for the most difficult object in the scene.

The ball detector used a YOLOv8x architecture and was trained on a dedicated Roboflow ball dataset. Training was run for 150 epochs with an image size of 1280 pixels, a batch size of 8, patience of 30 epochs, and caching enabled. The trained weights were saved as `models/ball_detection/best.pt`. During inference, this detector is used alongside the main detector, with additional temporal filtering applied later in the pipeline to reject implausible ball jumps.

3.4.3 Pitch Keypoint Dataset and Training

Pitch registration requires correspondences between points in the broadcast image and points on a canonical football pitch. To estimate these correspondences automatically, a YOLOv8x-pose model was trained to detect 32 canonical pitch keypoints. The model was trained using the Roboflow `football-field-detection` dataset, version 2, which contains one *pitch* class and 32 labelled keypoints. The training split contained 255 images and the validation split contained 34 images.

The pitch keypoint model was trained for 500 epochs with an image size of 640 pixels and a batch size of 48. Mosaic augmentation was disabled because geometric consistency between pitch markings is important for keypoint localisation and homography estimation. The resulting model weights were saved as `models/pitch_keypoints/best.pt`. At inference time, predicted keypoints are filtered by confidence before being used to estimate the homography.

3.4.4 Training Configuration

All YOLO models were trained using the Ultralytics framework. The default training configuration used automatic optimiser selection, an initial learning rate of 0.01, final learning-rate fraction of 0.01, momentum of 0.937, weight decay of 0.0005, warm-up for 3 epochs, automatic mixed precision, deterministic training, and seed 0 where supported.

Table 3.1 summarises the supervised models used in the system.

Model	Architecture	Task	Epochs	Image size
Main detector	YOLOv8m	Player/GK/referee/ball detection	50	1280
Ball detector	YOLOv8x	Ball detection	150	1280
Pitch keypoints	YOLOv8x-pose	32 pitch keypoints	500	640

Table 3.1: Summary of supervised models trained for the pipeline.

The trained weights are treated as external artefacts required for reproducibility. They are stored separately from the source code and must either be supplied with the project or regenerated using the documented training scripts and notebooks. Model-level validation results are reported in Chapter 4, while the effect of these models on the full pipeline is evaluated in Chapter 5.

3.5 Detection and Tracking

The first operational stage of the pipeline detects the visual entities required for downstream football analysis and assigns persistent identities to players across frames. This stage is critical because later components, including team identification, pitch-space projection, possession estimation, heatmaps, and offside candidate detection, all depend on the reliability of the detections and tracks produced here.

3.5.1 Main Object Detection

The main detector is a YOLOv8m object detection model trained to recognise four classes: players, goalkeepers, referees, and the ball. During inference, each video frame is passed through the detector at an image size of 1280 pixels. The runtime confidence threshold is set to 0.2 and the IoU threshold is set to 0.5. These values were chosen to retain sufficient detections for the tracker while still removing low-confidence false positives.

The detector output for each frame consists of a set of bounding boxes, class labels, and confidence scores. These detections are converted into supervision detection objects so that they can be passed consistently between the object detection, tracking, annotation, and tactical analysis modules. Preserving this intermediate representation also makes it easier to inspect and evaluate the behaviour of individual pipeline stages.

Although the main detector includes a ball class, the ball is substantially harder to detect than players because it occupies very few pixels and is often blurred or occluded. For this reason, the main detector is used primarily for players, goalkeepers, and referees, while ball localisation is strengthened using a separate dedicated detector.

3.5.2 Dedicated Ball Detection and Filtering

A dedicated YOLOv8x ball detector is applied to each frame to improve the reliability of ball localisation. This design decision was made because errors in ball detection propagate directly into possession, pass, and offside analysis. A missed or incorrect ball location can cause the system to assign possession to the wrong player, miss a pass transition, or trigger an incorrect offside candidate event.

The ball detector is run with a low initial confidence threshold (`ball_conf = 0.05`) so that weak ball candidates are not discarded too early. Candidate detections are then filtered temporally. A ball candidate is accepted only if it is sufficiently confident or spatially consistent with the previous accepted ball position. In particular, the pipeline uses a maximum jump threshold of 80 pixels and a minimum stronger-confidence threshold of 0.25. This prevents isolated low-confidence detections far from the previous ball position from being selected as the ball.

When multiple ball candidates are present, the system selects the candidate that is most consistent with the recent ball position. This filtering strategy does not solve all cases of ball occlusion or fast motion, but it reduces the effect of isolated false positives and produces a more stable ball trajectory for downstream tactical reasoning.

3.5.3 Multi-object Tracking

Player tracking is performed using tracking-by-detection. In each frame, the main YOLO detector supplies bounding boxes to a tracker, which assigns a persistent track identifier to each detected player. The default tracker used in the pipeline is ByteTrack, with BoT-SORT available as an alternative configuration. The tracker is selected through the runtime tracker configuration, allowing the same pipeline to be evaluated

with different tracking algorithms.

ByteTrack was selected as the default tracker because it is designed to use both high-confidence and lower-confidence detections during association. This is useful in broadcast football footage, where players are frequently partially occluded and their detection confidence may temporarily fall even when they remain visible. By retaining lower-confidence detections for a second association step, ByteTrack can reduce unnecessary track fragmentation in crowded scenes.

The tracker returns detections with associated track identifiers. These track identifiers are preserved in the pipeline and are used by later stages to accumulate player positions, smooth team labels, generate heatmaps, and compute player-level statistics. Detections without valid track identifiers are not used for player-level temporal statistics, since they cannot be linked reliably across frames.

3.5.4 Track Handling and Failure Cases

The tracking stage is vulnerable to several common failure cases. Occlusion can cause a player to disappear temporarily, leading to a broken track. Similar player appearance can cause identity switches, particularly when players cross or cluster around the ball. Camera motion can also affect the apparent movement of detections and make association more difficult.

The pipeline mitigates these issues in two ways. First, the tracker itself uses temporal association to preserve identities across short interruptions. Second, downstream stages avoid relying on a single frame wherever possible. For example, team labels are smoothed across a fixed track history, and movement statistics include a maximum-speed gate to reduce the effect of spurious projected movements.

Nevertheless, tracking errors cannot be fully eliminated at this stage. A fragmented or incorrect track can affect player heatmaps, distance estimates, team-level possession, and offside candidate detection. For this reason, the evaluation considers both the component-level behaviour of the tracker and the effect of tracking errors on the integrated pipeline.

3.6 Team Identification

After player detections have been assigned track identifiers, the next stage is to infer the team membership of each tracked player. This is required for team-level outputs such

as possession statistics, pass events, offside candidate detection, and team-coloured visualisations. The system performs team identification without match-specific manual labels by fitting an unsupervised classifier from player crops sampled from the input video.

3.6.1 Warm-up Sampling and Player Cropping

Team identification begins with a warm-up phase at the start of each video. During this phase, frames are sampled from the first 20 seconds of footage using a stride of 30 frames. The main detector is applied to these sampled frames, and only detections classified as players are retained. Very small crops are discarded using a minimum bounding-box size of 35 by 35 pixels, since such detections usually contain too little visual information for reliable team assignment.

For each retained player detection, the system extracts a torso crop rather than using the full bounding box. The crop is taken from approximately 15% to 65% of the bounding-box height. This focuses the representation on the shirt region, which usually contains the strongest team-specific visual signal, while reducing noise from shorts, legs, grass, and nearby players. A maximum of 800 warm-up crops is used, and at least 50 crops are required before the classifier is fitted.

3.6.2 Embedding and Clustering

The retained torso crops are used to fit a team classifier. The classifier extracts visual embeddings from each crop using a SigLIP-based image encoder, then reduces the embedding dimensionality using UMAP and clusters the reduced representations using K-means with $k = 2$. The two clusters correspond to the two outfield teams.

This approach was selected because it avoids requiring labelled team examples for every new match. A supervised team classifier would need new annotations whenever kits, lighting conditions, competitions, or broadcast styles changed. By fitting the classifier from the input video itself, the system can adapt to the visual appearance of the teams in the current match.

During the per-frame pipeline, each tracked player crop is passed through the same embedding and clustering pipeline to obtain a predicted team label. The output is a binary team assignment for outfield players. These assignments are then stabilised using temporal smoothing before being used by downstream analytics.

3.6.3 Temporal Smoothing and Role Handling

Frame-level team predictions can fluctuate because of occlusion, motion blur, partial crops, lighting variation, or players turning away from the camera. To reduce this instability, the system applies track-level majority-vote smoothing. For each track identifier, the most recent team predictions are stored in a fixed-length history of 30 predictions. The smoothed team label is then taken as the majority label within this history.

This design reduces the effect of isolated incorrect predictions while still allowing the team label to update if a track was initially misclassified. Smoothing is especially important because team labels are used in later components where a single-frame error can affect possession, pass, or offside logic.

Goalkeepers and referees require separate handling. Referees are assigned to a non-team class because they do not contribute to team possession or offside analysis. Goalkeepers are detected as a separate object class and are assigned to the team whose outfield-player centroid is nearest in image space. This spatial rule is used because goalkeeper kits often differ from the outfield players' kits, making appearance-based clustering less reliable for this role.

The resulting representation assigns each detection either to team 0, team 1, or a non-team class. This representation is used consistently by the visualisation, possession, pass, and offside modules.

3.7 Pitch Registration

Pitch registration maps image-space detections from the broadcast frame onto a canonical football pitch. This stage is required because image coordinates are affected by camera position, zoom, and perspective distortion. Tactical outputs such as heatmaps, speed, distance, possession proximity, and offside candidate detection therefore need to be computed in pitch space rather than directly in the broadcast image.

3.7.1 Pitch Keypoint Detection

Pitch landmarks are detected using a YOLOv8x-pose model trained to localise 32 canonical pitch keypoints. For each processed frame, the model predicts a set of keypoint coordinates and confidence scores. Keypoints with confidence below 0.5 are discarded before homography estimation. This filtering step reduces the effect of uncertain keypoint predictions, which can otherwise produce unstable or incorrect pitch

transformations.

A homography is estimated only when at least six valid keypoints are available. Although four point correspondences are theoretically sufficient to estimate a homography, requiring more than four points makes the transformation less dependent on any single detected landmark. Frames with too few reliable keypoints reuse the most recent valid transformation where possible, rather than forcing an unstable new estimate.

3.7.2 Homography Estimation and Coordinate Transformation

For each valid set of pitch keypoints, the system estimates a homography between the detected image-space landmarks and their corresponding locations on a canonical pitch model. The homography provides a projective transformation from image coordinates to pitch coordinates. Once this transformation is available, detected object positions can be expressed in pitch space.

For players, goalkeepers, and referees, the image-space position is taken as the bottom-centre point of the bounding box, since this approximates the point where the player contacts the ground. For the ball, the centre of the bounding box is used. These points are then transformed into pitch coordinates using the estimated homography.

The pitch coordinate system is represented in centimetres, with a pitch length of 12,000 cm. Using metric-like pitch units allows later stages to express distance thresholds and spatial rules in interpretable terms. For example, the possession and offside modules use distances in centimetres rather than pixel distances, making the thresholds less dependent on camera zoom.

3.7.3 Caching and Robustness Measures

Estimating a new pitch transformation on every frame is computationally expensive and can also introduce visual jitter when keypoint predictions vary slightly between consecutive frames. To reduce this problem, the pipeline recomputes the pitch transformation every 15 frames and caches the most recent valid transformation between updates.

This stride-cached design is a compromise between accuracy, stability, and runtime. Updating the homography too rarely can make the transformation lag behind camera motion, while updating it every frame can amplify keypoint noise. A stride of 15 frames provides regular updates while reducing unnecessary recomputation.

Pitch registration remains one of the most important sources of downstream error. If the homography is inaccurate, then all pitch-space outputs derived from it may be affected, including heatmaps, speed and distance estimates, possession proximity, and offside geometry. For this reason, registration quality is inspected both visually through bird’s-eye outputs and quantitatively where keypoint validation metrics are available.

3.8 Tactical Output Generation

Once detections have been tracked, assigned team labels, and transformed into pitch coordinates, the system generates tactical outputs from the resulting structured representation. These outputs are intended to make the broadcast video more analytically useful by converting low-level detections into visual summaries, player statistics, team possession estimates, pass events, and candidate offside situations.

3.8.1 Bird’s-eye Visualisation

The bird’s-eye visualisation represents the detected players, goalkeepers, referees, and ball on a canonical top-down pitch. For each frame with a valid pitch transformation, the image-space object positions are projected into pitch coordinates and rendered on the pitch model. Players are coloured according to their inferred team labels, referees are shown as a separate non-team class, and the ball is rendered using its filtered pitch-space position when available.

This visualisation provides an interpretable way to inspect whether the detection, team assignment, and pitch registration stages are behaving coherently. It is also used to generate side-by-side videos, where the original broadcast frame and the corresponding pitch-space representation are displayed together. This makes it easier to identify errors such as incorrect team labels, unstable homographies, or ball locations that do not match the broadcast footage.

3.8.2 Heatmaps, Speed and Distance

Player heatmaps are generated by accumulating each tracked player’s pitch-space positions over time. The pitch is divided into a fixed grid of 480×280 cells, and each valid player position contributes to the corresponding grid location. Heatmaps are generated

only when at least 40 valid samples are available for a track, reducing the likelihood of producing misleading visualisations from short or fragmented tracks.

To improve readability, the accumulated heatmap is smoothed using a Gaussian blur with kernel size 31 and a gamma correction of 0.22. The final heatmap is saved both as an image and as a raw numerical array, so that the visual output can be inspected while the underlying positional distribution remains available for further analysis.

Speed and distance estimates are computed from changes in player pitch-space position over time. Consecutive valid positions for each track are used to estimate displacement, which is accumulated to produce total distance travelled. Speed is derived from displacement and frame timing. Since small errors in pitch registration can produce unrealistically large apparent jumps, the system applies a maximum-speed gate of 40 km/h. Movements above this threshold are treated as noise and excluded from the accumulated statistics.

3.8.3 Possession and Pass Detection

Possession is estimated using the pitch-space distance between the filtered ball position and nearby players. For each frame, the system identifies the player closest to the ball. A player is considered a possession candidate only if the distance to the ball is below 350 cm. To reduce noise from brief misdetections or transient ball movements, possession must be stable for at least three consecutive frames before being confirmed.

Team possession is then obtained from the confirmed player's team label. If no player satisfies the proximity condition, the frame is treated as having no new confirmed possession event. This conservative design reduces the risk of assigning possession during periods when the ball is airborne, occluded, or incorrectly detected.

Pass events are detected from stable changes in possession. A pass is recorded when confirmed possession changes from one player to another in a temporally stable way. This rule-based definition is intentionally simple: it is designed to identify likely possession transfers rather than to provide a complete semantic classification of football actions. As a result, the pass output is best interpreted as a set of candidate pass events that require evaluation against manually inspected clips.

3.8.4 Offside Candidate Detection

Offside candidate detection is performed in pitch space using the detected players, inferred team labels, ball position, and estimated attacking direction. The attacking

direction is inferred automatically from accumulated team x -centroids once at least 30 valid team-position samples are available, rather than being manually specified for each video.

For each frame with sufficient information, the system estimates the offside line from the second-last defender’s pitch-space position. If only one defender is available, a fallback line is computed using the available defensive player, although this case is less reliable. An attacker is treated as a candidate offside player if they are beyond the defensive line by more than a 30 cm margin at the relevant moment and are spatially close enough to the ball event context to be considered involved in play.

The output of this module is not intended to replicate full official offside judgement. The official law depends on contextual factors such as the exact moment the ball is played, active involvement, deliberate play by defenders, and body-part eligibility. The system instead detects candidate offside situations under simplified geometric assumptions. Candidate events are saved as structured JSON records and visualised using an overlay for a fixed display duration of 50 frames, allowing them to be inspected during evaluation.

3.9 Evaluation Methodology

The evaluation was designed to assess both the individual components of the pipeline and the behaviour of the integrated system. This distinction is important because strong component-level performance does not necessarily imply that the final tactical outputs are reliable. For example, a detector may achieve high validation accuracy while still missing the ball in key moments, or a pitch registration model may perform well on isolated frames while producing unstable transformations during camera motion. The evaluation therefore considers not only model-level metrics, but also the extent to which the complete pipeline produces coherent and useful football analysis outputs.

3.9.1 Component-level Evaluation

The first level of evaluation assesses the supervised models used by the pipeline. The main object detector, dedicated ball detector, and pitch keypoint detector are evaluated using their validation or test splits, with metrics appropriate to each task.

For the object detection models, performance is measured using precision, recall, mAP@50, and mAP@50–95. These metrics assess both whether the relevant objects

are detected and how accurately their bounding boxes are localised. The dedicated ball detector is evaluated separately because ball localisation is a known bottleneck for downstream possession, pass, and offside analysis.

For the pitch keypoint model, evaluation considers both bounding-box and keypoint performance where available. The keypoint metrics are particularly important because pitch registration depends on the geometric accuracy of the detected landmarks rather than only on whether the pitch itself is detected.

Tracking is evaluated by comparing the stability and continuity of player tracks across representative clips. Where manual annotations are available, track fragmentation, identity switches, and track continuity are assessed directly. Where dense tracking ground truth is not available, the tracker is evaluated through visual inspection, track-count consistency, and its effect on downstream outputs such as heatmaps and distance estimates.

3.9.2 End-to-end Pipeline Evaluation

The second level of evaluation assesses the complete pipeline on broadcast football clips. This evaluation focuses on whether the system produces coherent outputs after all components have been combined. The main outputs considered are annotated broadcast videos, bird’s-eye visualisations, side-by-side videos, heatmaps, statistics files, and offside event records.

End-to-end evaluation is performed using representative clips containing common broadcast conditions, including camera panning, zooming, player occlusion, ball movement, and changes in pitch visibility. The outputs are inspected to assess whether detected entities remain visually consistent, whether team labels are stable, whether pitch-space projections remain aligned with the broadcast view, and whether the generated statistics are plausible.

Where manual labels are produced, tactical outputs are evaluated against manually inspected ground truth. Possession estimates are compared with the observed player or team in control of the ball, pass events are compared with manually identified possession transfers, and offside candidates are compared with visible attacking situations in the clip. This provides a more meaningful assessment of tactical usefulness than model-level metrics alone.

3.9.3 Ablation Studies

Ablation studies are used to evaluate the contribution of key design decisions. Each ablation changes one component or configuration while keeping the rest of the pipeline fixed, allowing the effect of that design choice to be isolated.

The planned ablations are:

- **Dedicated ball detector.** The pipeline is compared with and without the dedicated ball detector to determine whether the specialised model improves possession, pass, and offside outputs.
- **Tracker choice.** ByteTrack and BoT-SORT are compared on the same clips to assess differences in track continuity, identity stability, and downstream statistics.
- **Team-label smoothing.** Raw frame-level team predictions are compared with track-level majority-vote smoothing to assess whether smoothing reduces label instability.
- **Pitch registration stride.** Different homography update intervals are compared to measure the trade-off between runtime, projection stability, and responsiveness to camera motion.
- **Speed filtering.** Distance and speed estimates are compared with and without the maximum-speed gate to determine how much unrealistic movement is introduced by registration noise.

These ablations are included because the project contains several design choices that could otherwise appear arbitrary. Comparing alternatives makes it possible to justify the final configuration and identify which components have the greatest effect on end-to-end reliability.

3.9.4 Runtime Evaluation

Runtime evaluation measures the practical cost of processing broadcast video. The system records total processing time and, where possible, the time spent in major pipeline stages such as detection, ball detection, pitch registration, team assignment, rendering, and tactical output generation.

Runtime is reported as processing time per frame and effective frames per second. Additional comparisons are made for configurations that change runtime substantially, such as different pitch registration strides or optional bird’s-eye and side-by-side rendering. This allows the evaluation to distinguish between methods that improve output quality and methods that are too expensive for practical use.

The aim is not necessarily to achieve real-time performance, since the project prioritises analytical quality and reproducibility. However, runtime remains an important non-functional requirement because a system that is too slow to run on complete clips would have limited practical value.

3.9.5 Error Analysis

Error analysis is used to examine where the pipeline fails and how errors propagate between stages. This is necessary because the final tactical outputs depend on several earlier components. A missed ball detection can affect possession and pass detection; an identity switch can distort heatmaps and distance estimates; an incorrect team label can affect possession and offside logic; and an unstable homography can corrupt all pitch-space measurements.

The evaluation therefore records representative failure cases and groups them by cause. The main categories considered are missed or false detections, ball tracking errors, track fragmentation, team-label switches, pitch registration failures, and tactical inference errors. For each category, the analysis identifies the likely cause, the downstream effect, and any mitigation used in the pipeline.

This error-focused evaluation is important because the project is an integrated system rather than a single model. The objective is not only to report whether the outputs are correct, but also to explain why they succeed or fail under different broadcast conditions. This provides the basis for the critical evaluation in Chapter 5.

3.10 Implementation and Reproducibility

The pipeline was implemented in Python, using a modular code structure so that individual stages could be configured, tested, and replaced independently. The main entry point is `src/track_video_supervision.py`, which parses the runtime configuration and calls the pipeline orchestration code in `src/pipeline/run.py`. This organisation

separates high-level execution from the component-specific modules used for detection, team identification, pitch registration, visualisation, statistics, and offside detection.

3.10.1 Software Environment

The implementation uses Python 3.11. The main libraries are Ultralytics for YOLO model training and inference, Supervision for detection handling and video annotation utilities, OpenCV for image processing and video writing, NumPy for numerical operations, scikit-learn and UMAP for clustering-related operations, and Hugging Face Transformers for the SigLIP-based team embedding model. The Roboflow inference tools are used for the pitch model integration, and the Roboflow sports library is used for the team classification pipeline.

The project dependencies are recorded in `requirements.txt`. In addition, the Roboflow sports library is installed as an editable dependency from its source repository. The environment also requires a Roboflow API key and field model identifier, which are specified through environment variables. An example environment file is provided to document the required variables without exposing private credentials.

3.10.2 Configuration and Model Weights

The pipeline is controlled through runtime parameters rather than hard-coded settings. Important configuration values include the detector image size, detection confidence threshold, IoU threshold, tracker type, pitch registration stride, keypoint confidence threshold, ball filtering thresholds, team smoothing window, heatmap settings, and offside detection parameters.

The default inference configuration uses an image size of 1280 pixels, detector confidence threshold of 0.2, IoU threshold of 0.5, and ByteTrack as the tracker. The team classifier samples the first 20 seconds of video with a stride of 30 frames and applies a 30-frame smoothing window to team predictions. Pitch registration is updated every 15 frames, using keypoints with confidence at least 0.5 and requiring at least six valid keypoints before estimating a homography.

The trained model weights are stored separately from the source code. The required weights are the main object detector `models/object_detection/best.pt`, the dedicated ball detector `models/ball_detection/best.pt`, and the pitch keypoint detector `models/pitch_keypoints/best.pt`. To reproduce the full system, these weights

must either be supplied with the project artefacts or regenerated using the documented training scripts and notebooks.

3.10.3 Running the Pipeline

The pipeline can be executed from the command line using the provided run scripts. The standard execution path processes an input video, writes an annotated output video, and optionally generates bird’s-eye videos, side-by-side videos, heatmaps, statistics files, pitch debug videos, and offside event records. The main generated outputs are written to the `outputs/` directory.

For each input video, the core output is an annotated video showing detections, track identifiers, team labels, the ball marker, and offside overlays where enabled. Optional outputs include a bird’s-eye video, a side-by-side broadcast-and-pitch view, JSON statistics, JSON offside-event records, and per-player heatmap files. The statistics output records team possession, passes, player speed estimates, and distance travelled, while the heatmap directory stores both image files and raw numerical arrays.

This output structure was chosen to support both visual inspection and later evaluation. Videos make it possible to inspect whether the integrated pipeline is behaving plausibly, while JSON and numerical outputs allow quantitative comparison with manual labels and ablation results.

3.10.4 Reproducibility Limitations

Several limitations affect exact reproducibility. First, the trained model weights are external artefacts and are not stored directly inside the source code repository. Reproducing the system therefore requires either access to the trained weights or re-training the models from the documented datasets and training configuration.

Second, some datasets and hosted models are accessed through Roboflow services, so reproduction depends on access to the relevant workspaces, dataset versions, and API credentials. Third, although YOLO training used deterministic settings and a fixed seed where supported, the runtime team classifier is fitted from sampled video crops and may vary if the underlying embedding, UMAP, or clustering behaviour is not fully deterministic.

Finally, the current implementation is primarily an experimental research pipeline

rather than a fully packaged software product. It provides scripts, configuration parameters, model artefact locations, and documented outputs, but does not include a complete automated unit-test suite. This limitation is addressed in the evaluation through component-level validation, visual inspection, ablation studies, and end-to-end testing on representative clips.

3.11 Chapter Summary

This chapter described the methodology used to construct the proposed broadcast football video analysis pipeline. The system was designed around a set of functional and non-functional requirements, with the central objective of producing tactical outputs from a single broadcast video without relying on commercial tracking infrastructure.

The methodology used a modular architecture in which object detection, ball detection, multi-object tracking, team identification, pitch registration, and tactical output generation are implemented as separable stages. This structure allows each component to be configured and evaluated independently, while still supporting end-to-end analysis of the complete pipeline.

The chapter also described the datasets, model training procedures, runtime configuration, and reproducibility measures used by the system. Particular attention was given to design decisions that affect downstream reliability, including the use of a dedicated ball detector, track-level team-label smoothing, stride-cached pitch registration, and pitch-space rules for possession, pass, and offside candidate detection.

The following chapter reports the results produced by this methodology, including model-level validation results, pipeline outputs, tactical statistics, visualisations, and runtime measurements. Chapter 5 then evaluates these results against the requirements and success criteria defined in this chapter.

Chapter 4

Results

4.1 Experimental Setup

4.2 Findings

Chapter 5

Evaluation

5.1 Analysis

5.2 Discussion

5.3 Limitations

Chapter 6

Conclusion

6.1 Summary

6.2 Future Work

Bibliography

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory (ICDT)*, pages 420–434. Springer, 2001.
- [2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.
- [3] S. Altmann, L. Forcher, L. Ruf, A. Beavan, T. Groß, P. Lussi, and M. Grossniklaus. Match-related physical performance in professional soccer: Position or player specific? *PLOS ONE*, 16(9):e0256695, 2021.
- [4] M. Beetz, S. Gedikli, J. Bandouch, B. Kirchlechner, N. von Hoyningen-Huene, and A. Perzylo. Visually tracking football games based on tv broadcasts. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [5] A. Cioppa, S. Giancola, A. Delière, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022.
- [6] J. Davis, L. Bransen, L. Devos, A. Jaspers, W. Meert, P. Robberechts, J. Van Haaren, and M. Van Roy. Methodology and evaluation in sports analytics: challenges, approaches, and lessons learned. *Machine Learning*, 113:6977–7010, 2024.
- [7] A. Delière, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasber, B. Ghanem, and M. Van Droogenbroeck. SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos. In *Proceedings of*

the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops, pages 4508–4519, 2021.

- [8] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixe. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021.
- [9] S. Ellens, D. Hodges, S. McCullagh, J. J. Malone, and M. C. Varley. Interchangeability of player movement variables from different athlete tracking systems in professional soccer. *Science and Medicine in Football*, 6(1):1–6, 2022.
- [10] J. Fernandez and L. Bornn. Wide open spaces: A statistical technique for measuring space creation in professional soccer. In *MIT Sloan Sports Analytics Conference*, 2018.
- [11] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [12] F. R. Goes, L. A. Meerhoff, M. J. O. Bueno, D. M. Rodrigues, F. A. Moura, M. S. Brink, M. T. Elferink-Gemser, A. J. Knobbe, S. A. Cunha, R. S. Torres, and K. A. P. M. Lemmink. Unlocking the potential of big data to support tactical performance analysis in professional soccer: A systematic review. *European Journal of Sport Science*, 21(4):481–496, 2021.
- [13] A. Gualtieri, E. Rampinini, A. Dello Iacono, and M. Beato. High-speed running and sprinting in professional adult soccer: Current thresholds definition, match demands and training strategies. a systematic review. *Frontiers in Sports and Active Living*, 5:1116293, 2023.
- [14] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.
- [15] M. Istasse, J. Moreau, and C. De Vleeschouwer. Associating players to teams using visual features trained with weak supervision. In *International Conference on Machine Vision Applications (MVA)*, 2019.
- [16] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023.

- [17] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [18] M. Kisantal, Z. Wojna, J. Muber, A. Pirinen, and E. Rahtu. Augmentation for small object detection. In *9th International Conference on Advanced Concepts for Intelligent Vision Systems (ACIVS)*, 2019. arXiv preprint arXiv:1902.07296.
- [19] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [21] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [22] D. Linke, D. Link, and M. Lames. Football-specific validity of TRACAB’s optical video tracking systems. *PLOS ONE*, 15(3):e0230179, 2020.
- [23] D. Linke, D. Link, and M. Lames. Validation of electronic performance and tracking systems EPTS under field conditions. *PLOS ONE*, 15(7):e0236831, 2020.
- [24] J. Liu, X. Tong, W. Li, T. Wang, Y. Zhang, and H. Wang. Automatic player detection, labeling and tracking in broadcast soccer video. *Pattern Recognition Letters*, 30(2):103–113, 2009.
- [25] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37. Springer, 2016.
- [26] L. Lolli, P. Bauer, C. Irving, D. Bonanno, O. Höner, W. Gregson, and V. Di Salvo. Data analytics in the football industry: a survey investigating operational frameworks and practices in professional clubs and national federations from around the world. *Science and Medicine in Football*, 2024.
- [27] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.

- [28] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.
- [29] F. Magera, T. Hoyoux, O. Barnich, and M. Van Droogenbroeck. BroadTrack: Broadcast camera tracking for soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Tucson, Arizona, USA, February 2025.
- [30] A. Maglo, A. Orcesi, J. Denize, and Q. C. Pham. Individual locating of soccer players from a single moving view. *Sensors*, 23(18):7938, 2023.
- [31] L. McInnes, J. Healy, N. Saul, and L. Großberger. Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.
- [32] T. Meinhardt, A. Kirillov, L. Leal-Taixé, and C. Feichtenhofer. TrackFormer: Multi-object tracking with transformers. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8844–8854, 2022.
- [33] X. Nie, S. Chen, and S. Agarwal. Robust and efficient sports field registration. In *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2021. Replace with your preferred direct-regression registration reference if different.
- [34] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [35] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [36] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [37] J. Redmon and A. Farhadi. YOLO9000: Better, faster, stronger. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6517–6525, 2017.

- [38] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [39] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [40] V. Somers, V. Joos, A. Cioppa, S. Giancola, S. A. Ghasemzadeh, F. Magera, B. Standaert, A. M. Mansourian, X. Zhou, S. Kasaei, B. Ghanem, A. Alahi, M. Van Droogenbroeck, and C. De Vleeschouwer. Soccernet game state reconstruction: End-to-end athlete tracking and identification on a minimap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3293–3305, June 2024.
- [41] The International Football Association Board. Laws of the game 2025/26. <https://downloads.theifab.com/downloads/laws-of-the-game-2025-26-single-pages?l=en>, 2025. Accessed 2026-04-08.
- [42] J. Theiner and R. Ewerth. Tvcilib: Camera calibration for sports field registration in soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1166–1175, 2023.
- [43] J. Theiner, W. Gritz, E. Müller-Budack, R. Rein, D. Memmert, and R. Ewerth. Extraction of positional player data from broadcast soccer videos. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1463–1473, 2022.
- [44] F. Vidal-Codina, N. Evans, B. El Fakir, and J. Billingham. Automatic event detection in football using tracking data. *Sports Engineering*, 25(1), 2022.
- [45] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.
- [46] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, volume 13682 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2022.

Appendix A

Additional Materials